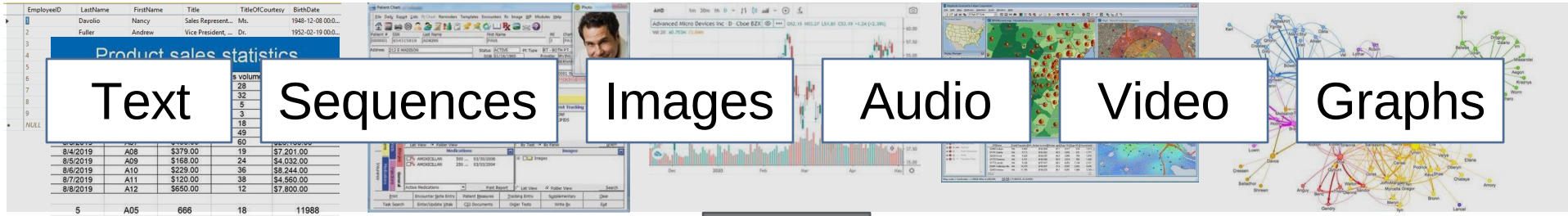
向量存储系统

Slides来源sigmod’24 tutorial
以及Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD, OSDI’25

- Vector search index
- Scalable vector search systems
- Commercial vector systems
- Challenges and open problems

Embeddings: Building Blocks of the Future

More and more applications rely on deep-learning **embedding vectors** that can only be understood by machines



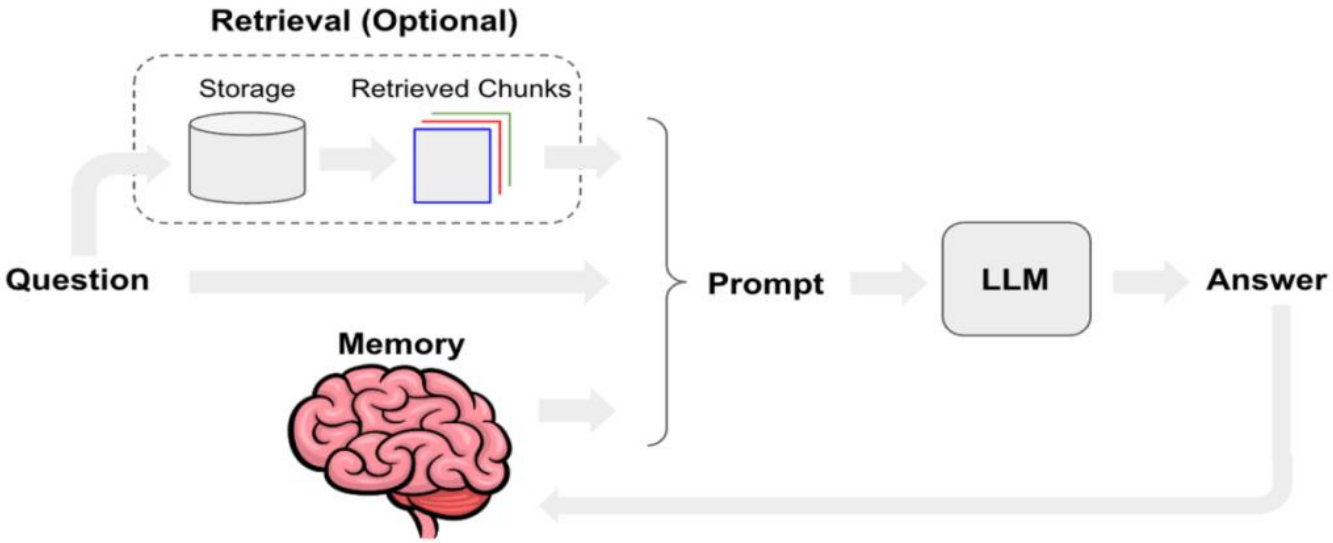
Text | Sequences | Images | Audio | Video | Graphs

DeepMind | Hugging Face | OpenAI

MiniLM-L6 | MPNet-Base | ResNet-50 | CLIP VIT | Whisper | Multimodal Receiver

Forecasting | Classification | RecSys | Pattern Recognition | Reinforcement Learning | Object Detection | Semantic Retrieval | Segmentation | Generation

Agent memory

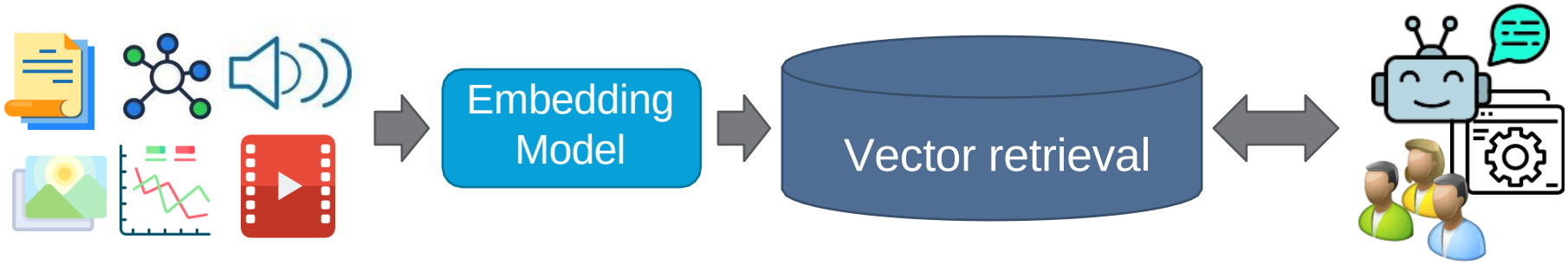


记忆工具

OpenClaw 为这些 Markdown 文件提供了两个面向智能体的工具

- `memory_search` — 对索引片段进行语义检索。
- `memory_get` — 对特定 Markdown 文件/行范围进行定向读取。

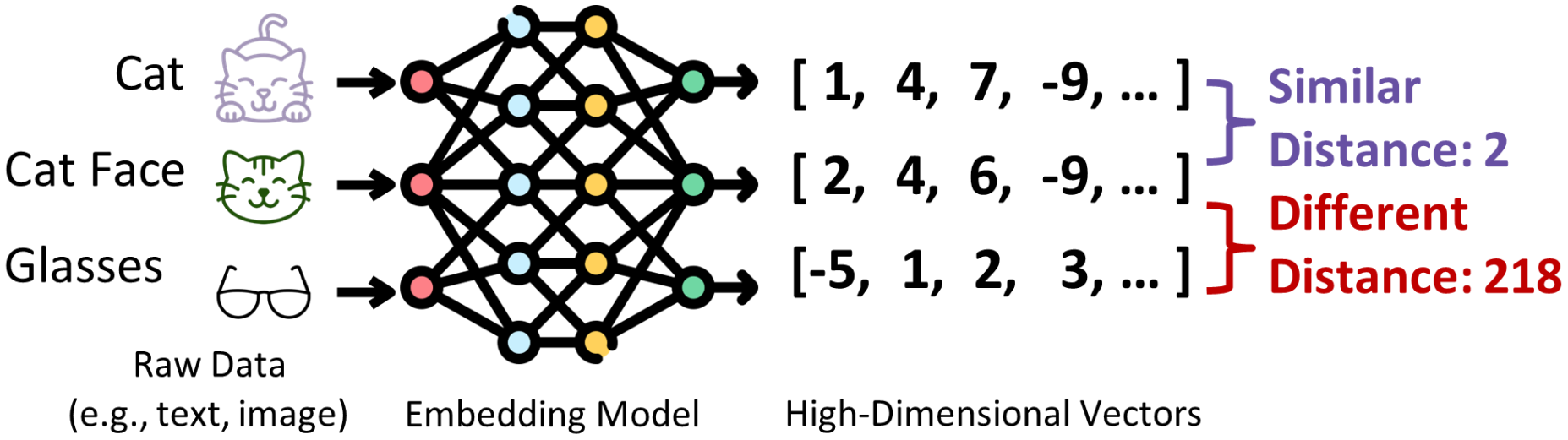
Goal: Vector retrieval



Store embeddings and retrieve desired embeddings for whatever downstream task

- Capabilities: similarity-based top-k/range retrieval, hybrid attribute-vector retrieval, multi-modal (multi-vector) retrieval
- Characteristics: read/write latency/throughput, retrieval accuracy, scalability, availability, consistency, fault tolerance, privacy & security, elasticity

Goal: Vector retrieval



Goal: Vector retrieval

Vector dataset

[1, 4, 7, -9, …]

[2, 4, 6, -9, …]

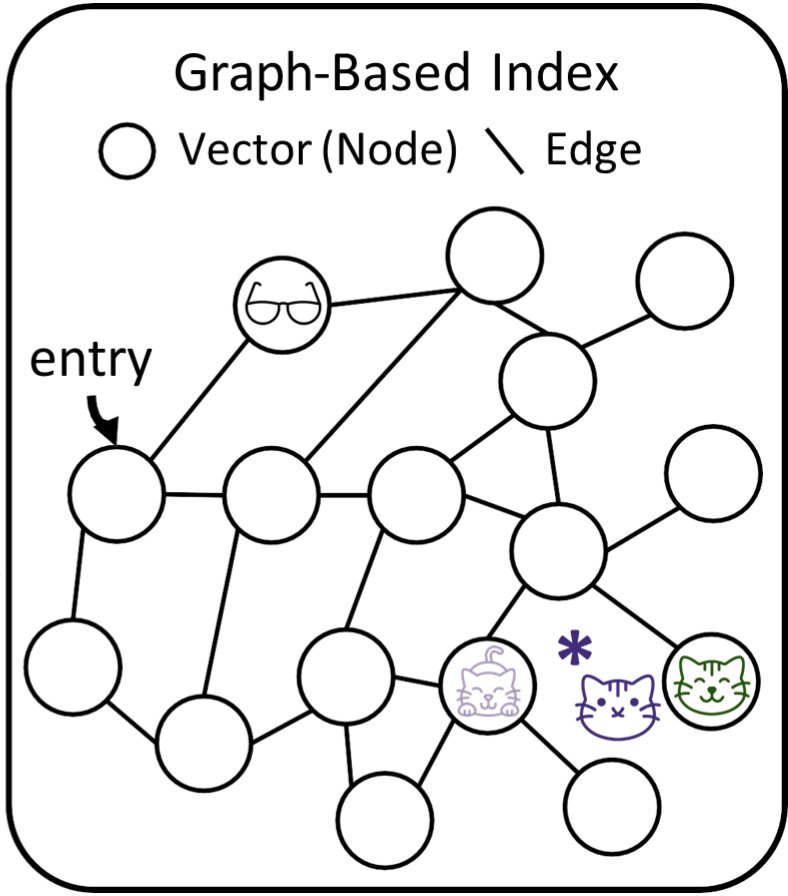
[-5, 1, 2, 3, …]

Build

Target (Query)

[3, 5, 7, -8, …]

Search



Accurate search (e.g., brute-force) is slow

Complexity: O(#vectors × #dims)

So, use approximate search (ANNS) instead

Goal: Vector retrieval

Vector dataset


[1, 4, 7, -9, …]


[2, 4, 6, -9, …]

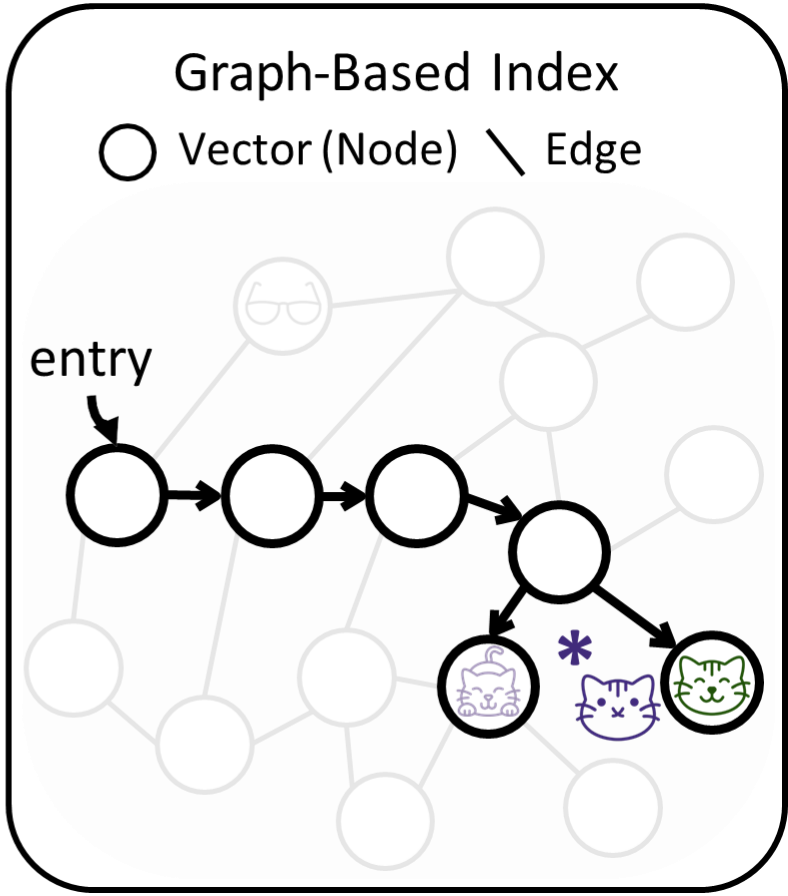

[-5, 1, 2, 3, …]

Build →

Target (Query)


[3, 5, 7, -8, …]

Search →

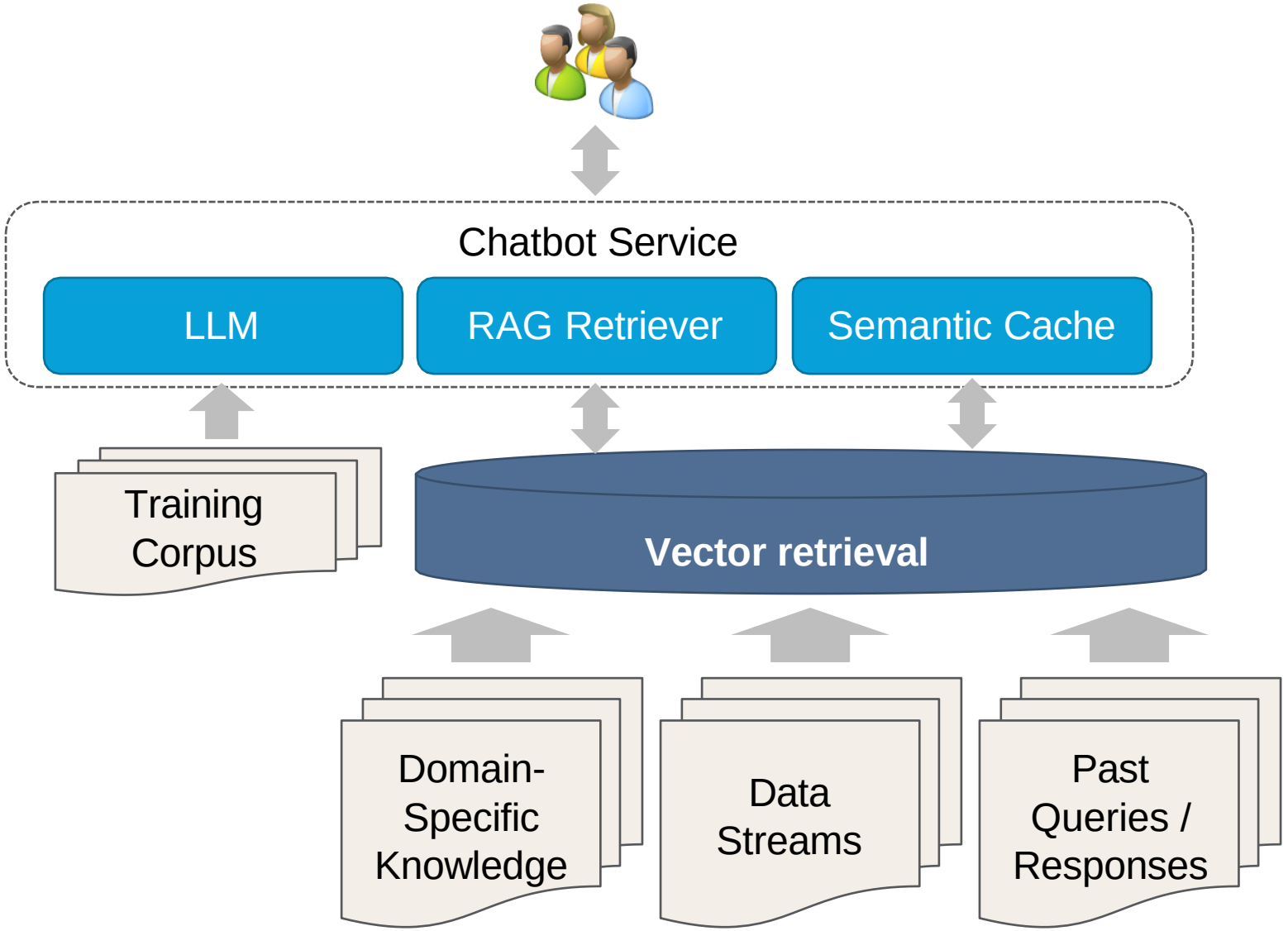


Accurate search (e.g., brute-force) is slow

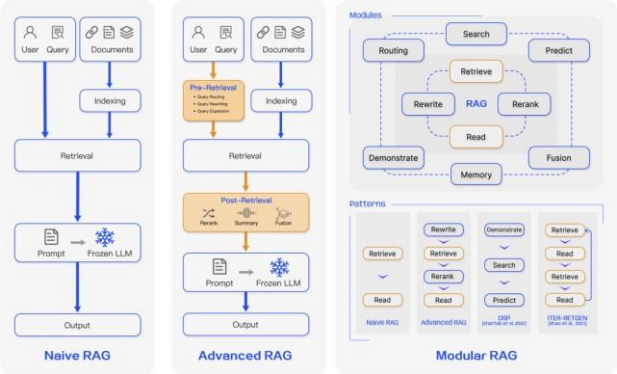
Complexity: O(#vectors × #dims)

So, use approximate search (ANNS) instead

Example: LLMs + Vector retrieval



Some of Today's Commercial Applications



LLM Retrieval-Augmented Generation (RAG)

京东 汽车之家 看车·买车·用车 小红书 趣头条 QUTOUTIAO iQIYI 爱奇艺 Shopee

E-Commerce & Recommendation Systems

Writing Assistant

搜狐

News Classification

Likee picmo

Photo/Video Search & Deduplication

TREND MICRO

Threat Detection

Vector retrieval is hard?

Embeddings are...

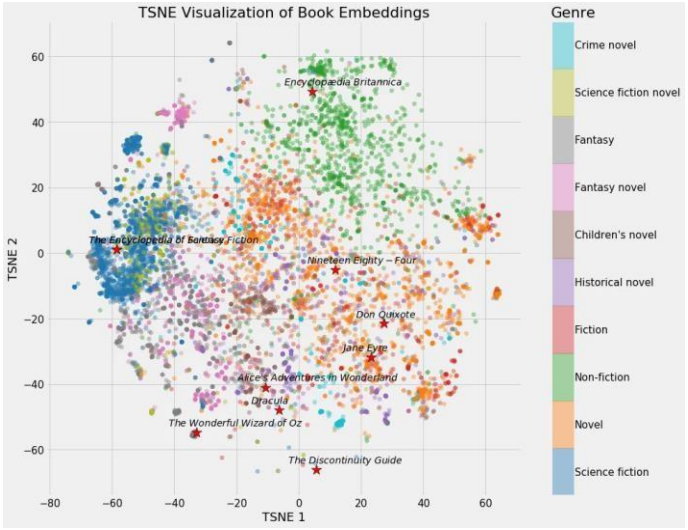


Figure: Will Koehrsen

vs

EmployeeID	LastName	FirstName	Title	TitleOfCourtesy	BirthDate

Product sales statistics

Date	Product name	Price	Sales volumes	Sales amount
7/28/2019	A01	$584.00	28	$16,352.00
7/29/2019	A02	$369.00	32	$11,808.00
7/30/2019	A03	$152.00	5	$760.00
7/31/2019	A04	$879.00	3	$2,637.00
8/1/2019	A05	$666.00	18	$11,988.00
8/2/2019	A06	$288.00	49	$14,112.00
8/3/2019	A07	$436.00	60	$26,160.00
8/4/2019	A08	$379.00	19	$7,201.00
8/5/2019	A09	$168.00	24	$4,032.00
8/6/2019	A10	$229.00	36	$8,244.00
8/7/2019	A11	$120.00	38	$4,560.00
8/8/2019	A12	$650.00	12	$7,800.00
5	A05	666	18	11988

- Huge (1024 x float64) → costly to move, clog storage
- Hard to retrieve without ambiguity
- Hard to index
- Costly to compare
- Hard to index together with attributes

similar to

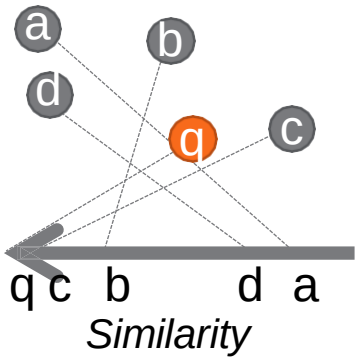
Query Definition: Similarity Scores

Similarity Score

- Inner Product, Cosine Similarity

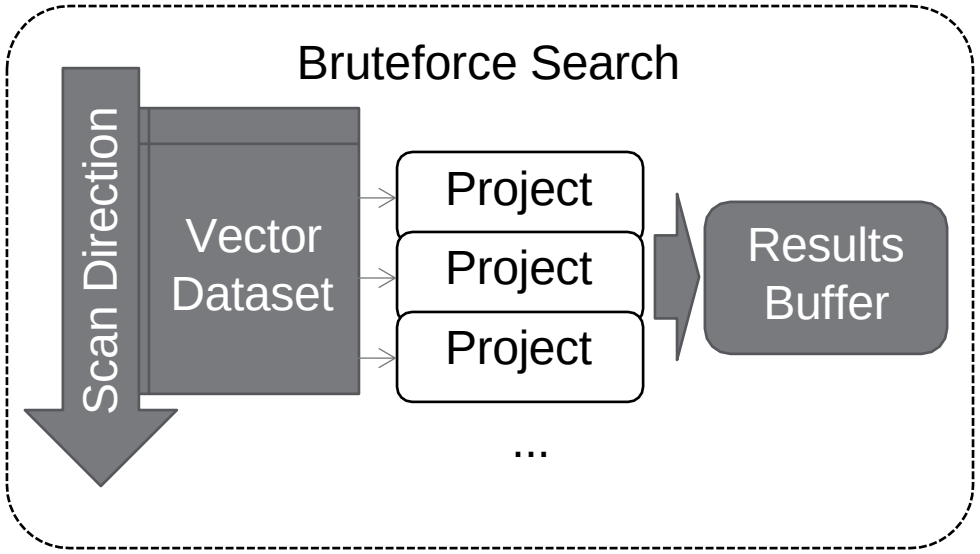
- A function $f: \mathbb{R}^D \times \mathbb{R}^D \rightarrow \mathbb{R}$ indicating degree of similarity
- Similarity calculations are expensive
  - Sq. Euclidean (D=1024 floats) takes 62us on my machine (Intel i5 @ 2.3 GHz), about the same as SSD random seek

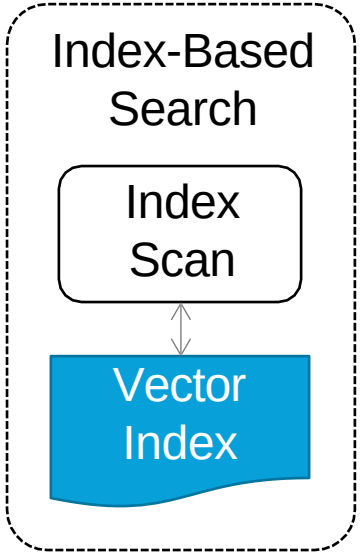
Operators & Algorithms

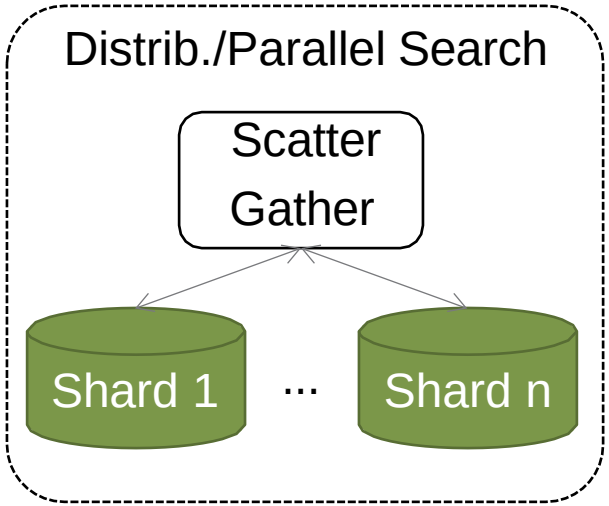


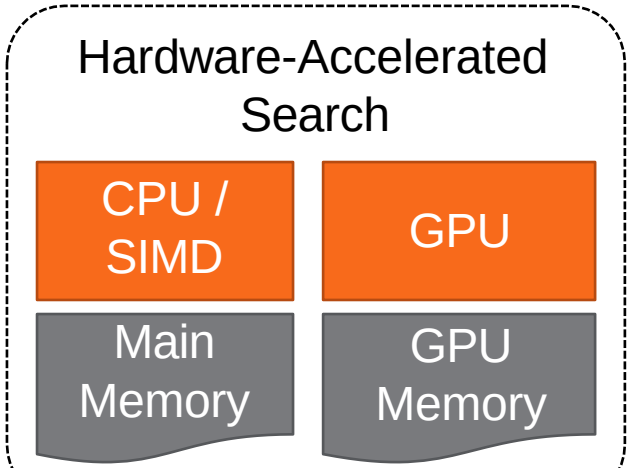
Project a vector onto similary score with respect to query vector

- O(D) for D-dimensions









Characteristics of Search Algorithms

Performance

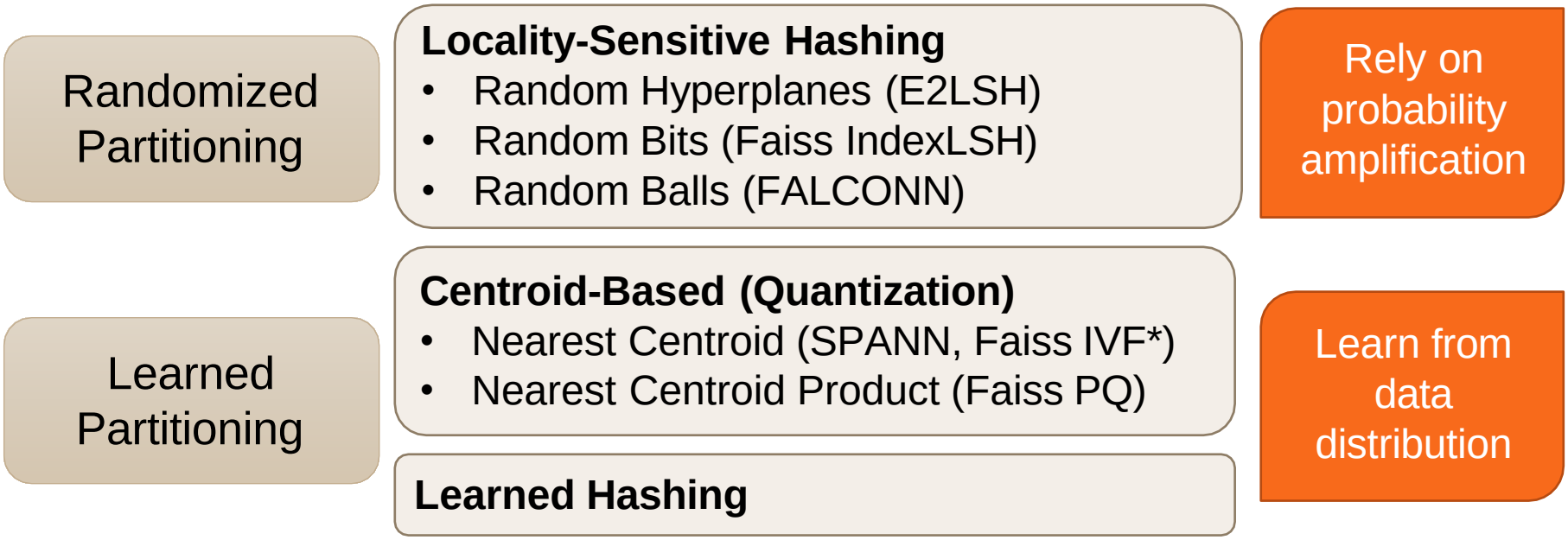
- Amount of visited vectors, similarity comparisons

Accuracy

- Recall: (true positives) / (true positives + false negatives)
- Recall@K
  - Recall when k=K for k-NN, ANN (Li et al 2020)
  - Proportion of queries where 1-NN is ranked in first k results (Jegou et al 2011)
  - Proportion of true nearest-neighbors within the first K results of a k-NN or ANN query (K ≤ k) (RecSys)
- Precision: (true positives) / (true positives + false positives)

1. Vector search index

Table-Based Indexes

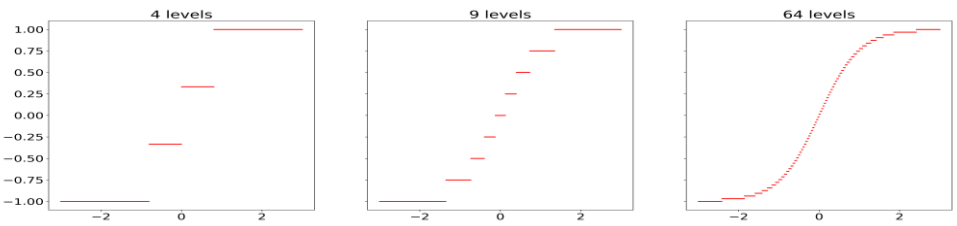


Quantization

Level Quantization

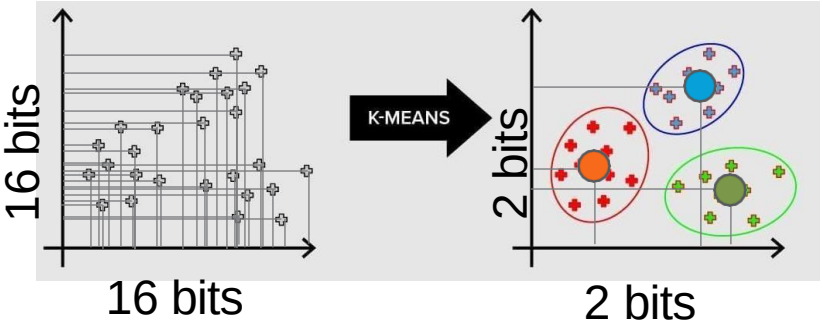
- Uniform (e.g. Faiss SQ)
- Non-Uniform

2 bits | 4 bits | 6 bits



Learned Quantization

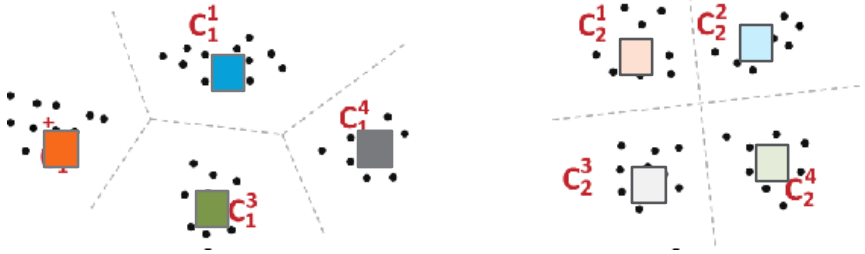
- k-Means (e.g. Faiss IVFSQ)
- Product (e.g. Faiss PQ)



E.g. 4-Dimensional Vectors m=2, k=4

Dims [1, 2] → subspace 1

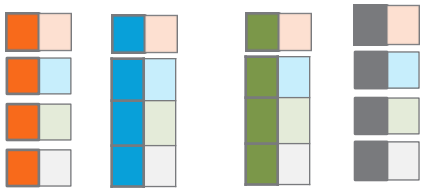
Dims [3, 4] → subspace 2



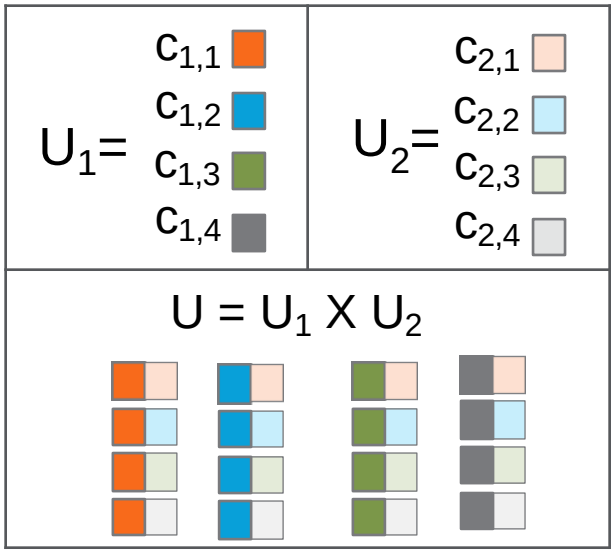
$U_1 = c_{1,1}, c_{1,2}, c_{1,3}, c_{1,4}$

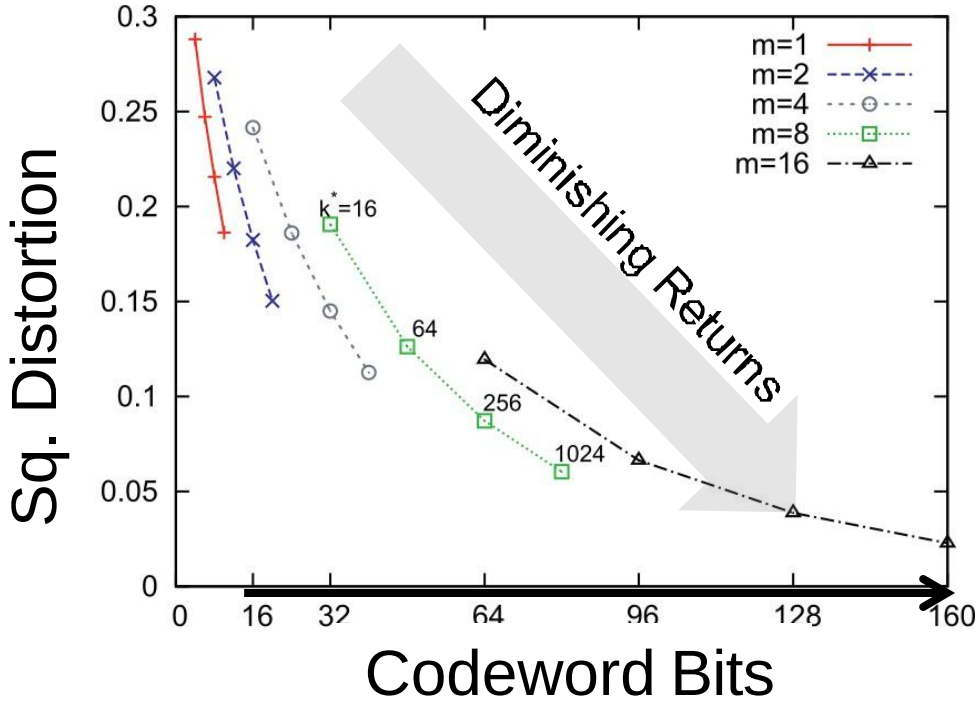
$U_2 = c_{2,1}, c_{2,2}, c_{2,3}, c_{2,4}$

$U = U_1 \times U_2$



Product Quantization





- Preserves dimensions, e.g. $R^4 = R^2$ X R^2
- Faster training
  - 4 centroids per subspace = 16 total codes

Figure: Jegou et al 2011

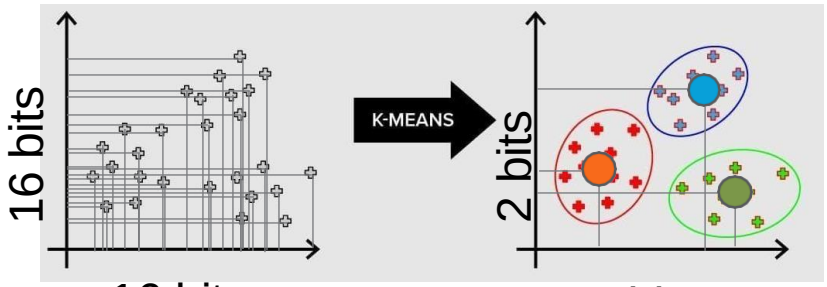
“IVFADC” Asymmetric Distance Comp.

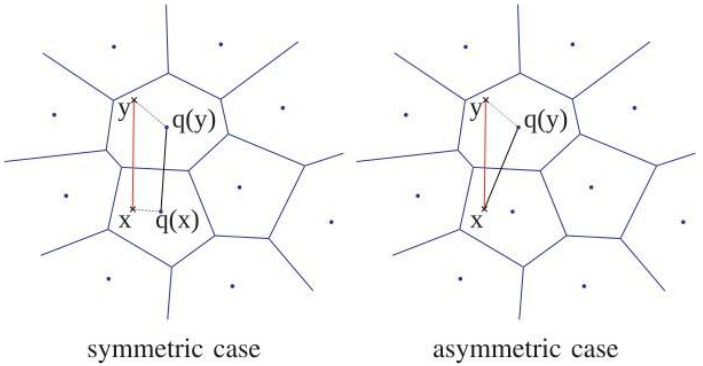
Index

Search

Buckets

Coarse Quantization





Compressed Entries

Vector Centering

Vector Residuals

Sample + PQ

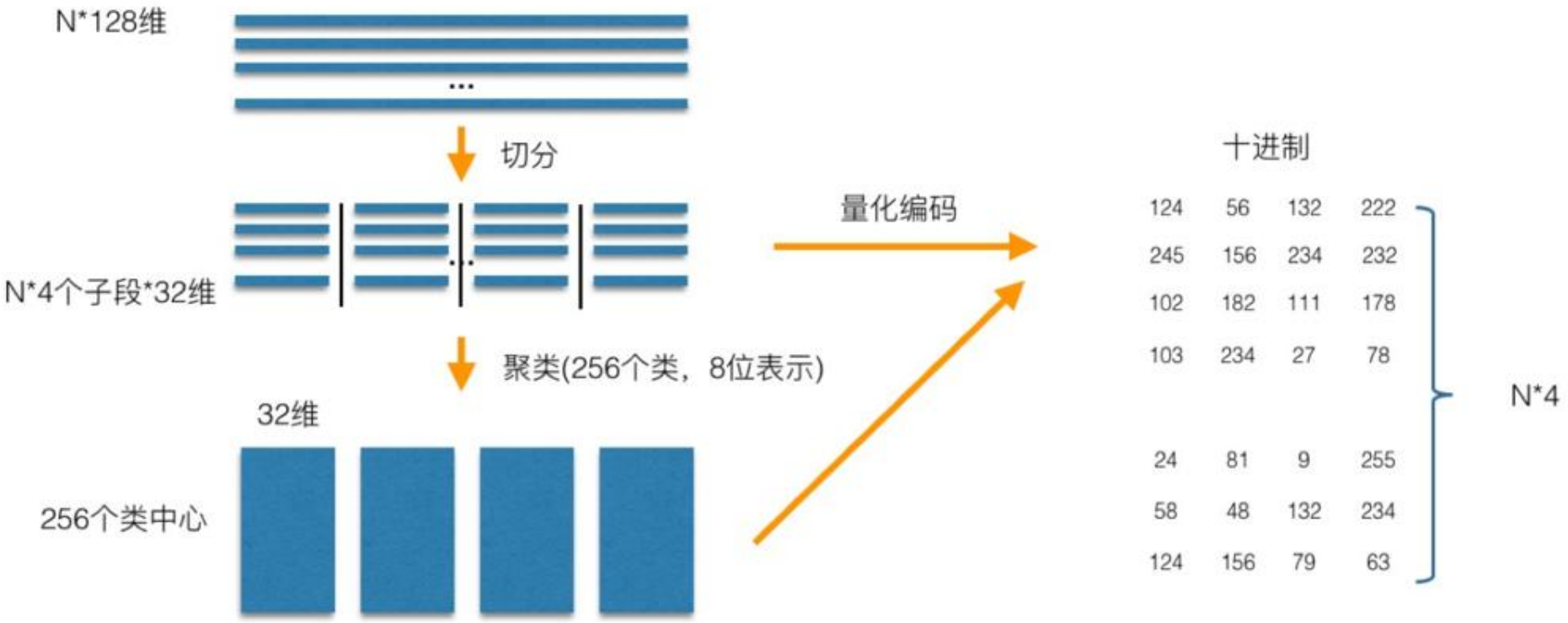
Codebook

ADC Lookup Table

$$\overbrace{\begin{matrix} d(\overline{\mathbf{q}_1}, \mathbf{c}_1^m) & \cdots & d(\overline{\mathbf{q}_1}, \mathbf{c}_{K'}^m) \\ \vdots & \ddots & \vdots \\ d(\overline{\mathbf{q}_m}, \mathbf{c}_1^m) & \cdots & d(\overline{\mathbf{q}_m}, \mathbf{c}_{K'}^m) \end{matrix}}^{U_m}$$

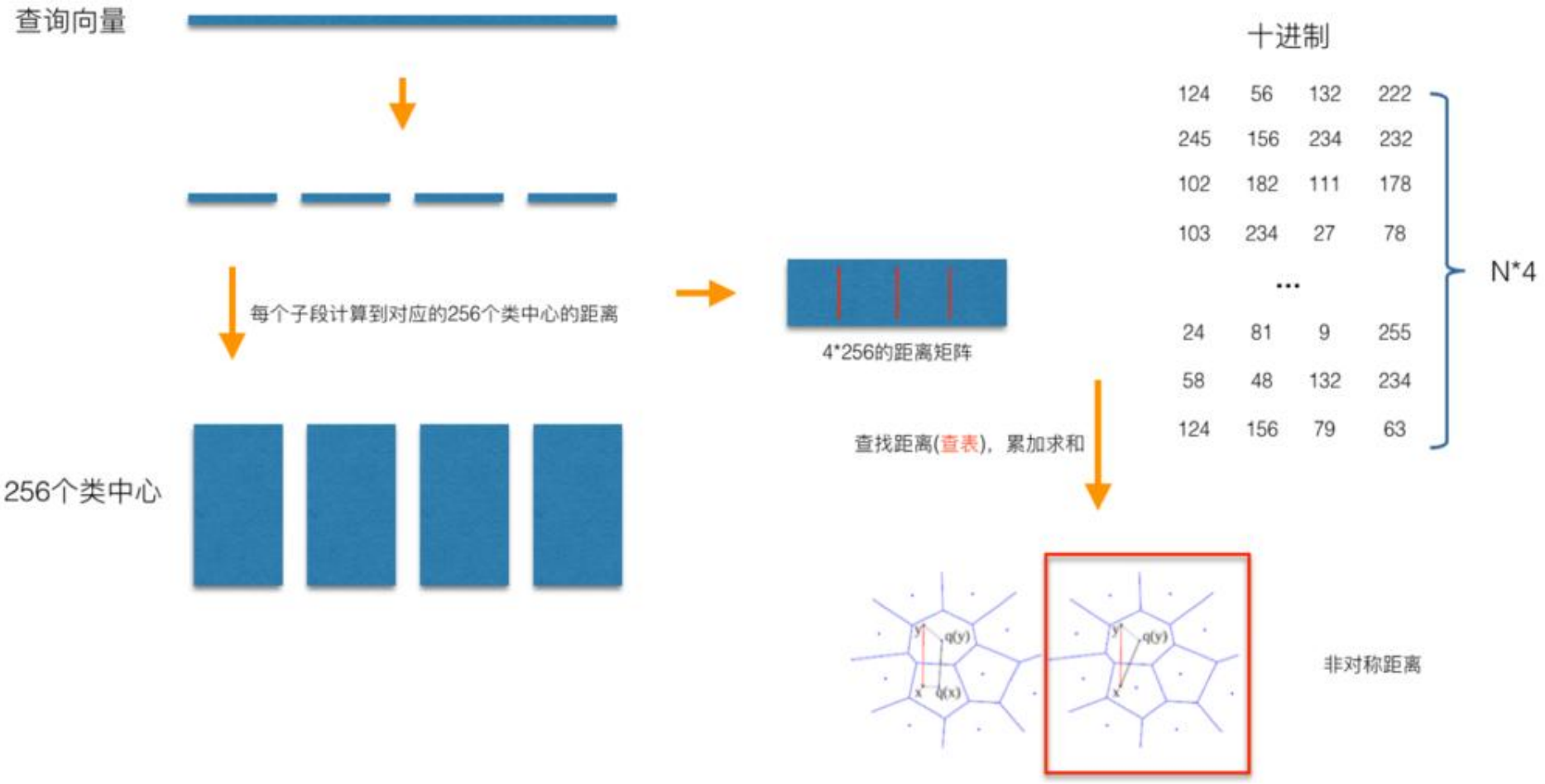
- Bucketing for fast search
- Lookup cache
- Sampling from residuals for faster training

PQ & IVFADC



每个vector可以使用量化编码表示

PQ & IVFADC



Query切分，每个子段与类中心进行距离计算

如果query要与成千上万个向量进行计算，怎么加速？

Tree-based index：k-d Tree

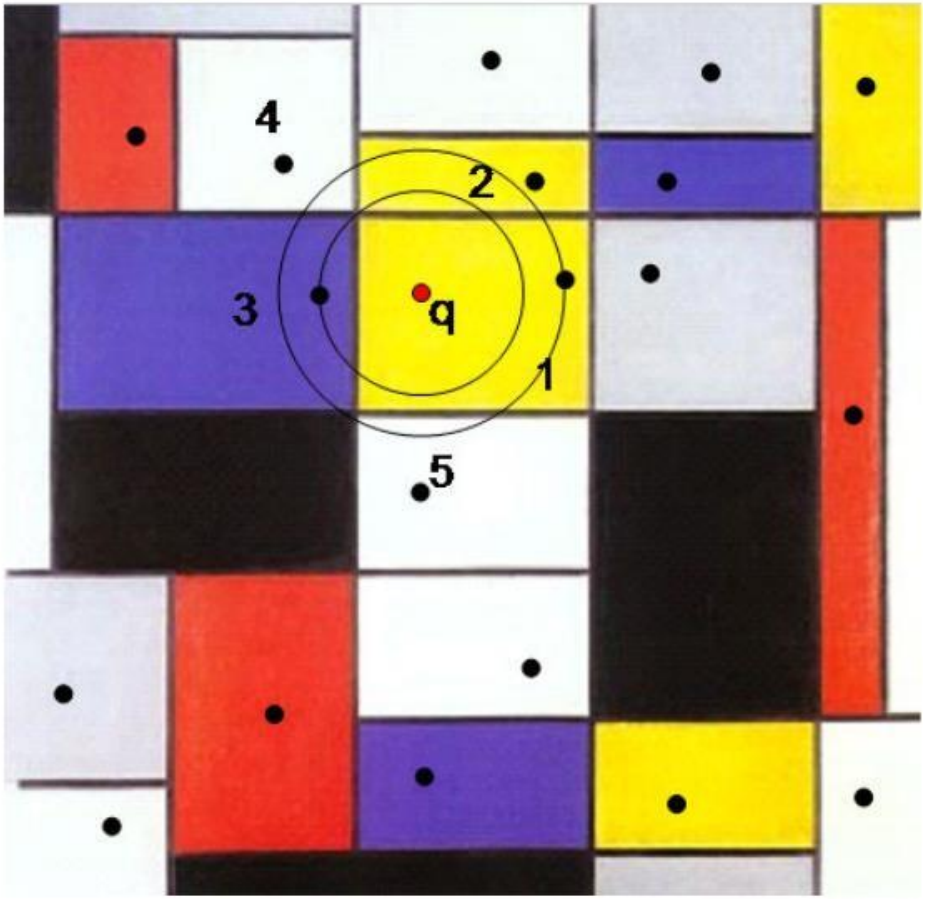


Figure: Silpa-Anan and Hartley, 2008

Graph-Based Indexes

Navigable Partitioning

k-Nearest Neighbor Graphs (kNNGs)
- KGraph
- EFANNA

Directly index the nearest neighbors

Monotonic Search Networks (MSNs)
- FANNG
- NSG
- Vamana

Support greedy depth-first search

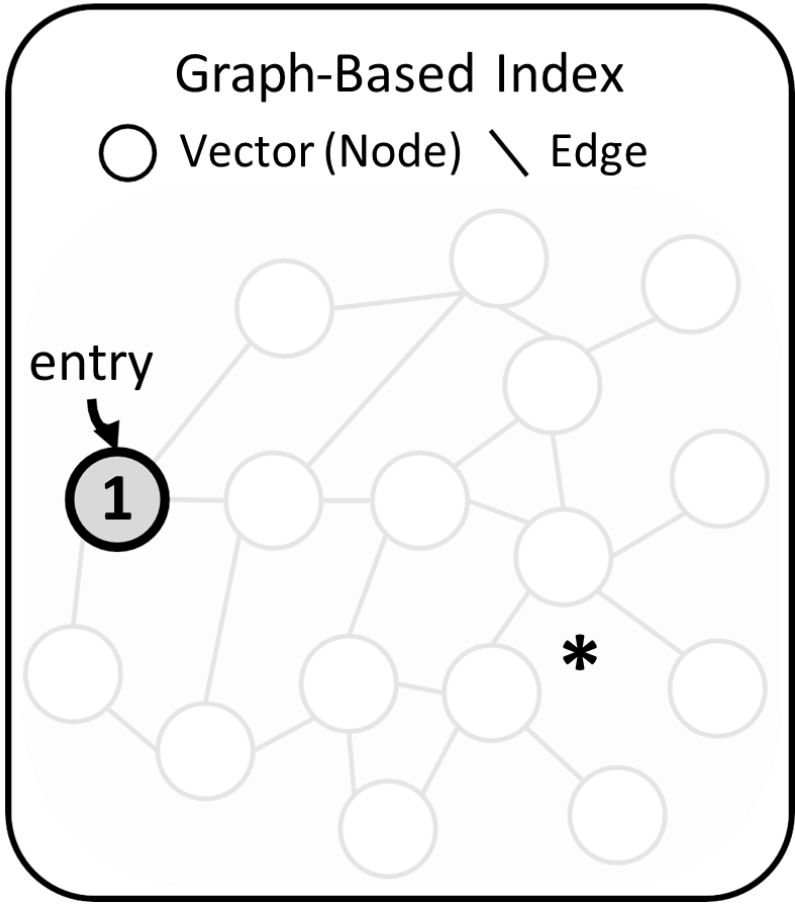
Small-World Graphs
- NSW
- HNSW

Exploit navigational small-world properties

Best-First Search Algorithm

Candidate pool: **{ 1 }**

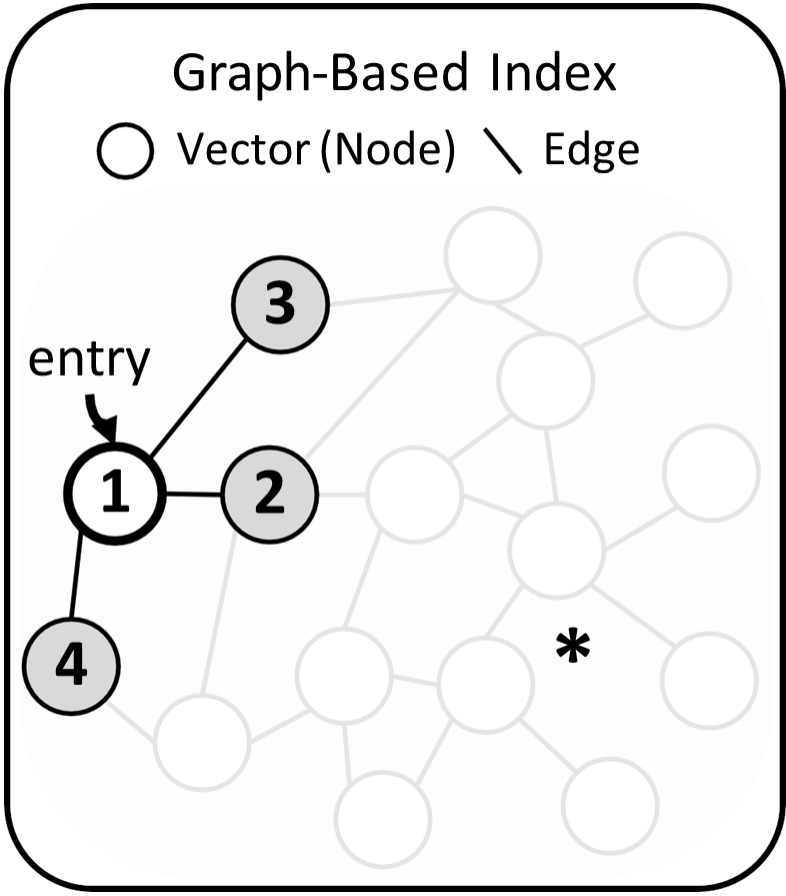
- **Select** the nearest unvisited vector **1** in the *candidate pool*



Best-First Search Algorithm

Candidate pool: **{ 1 }**

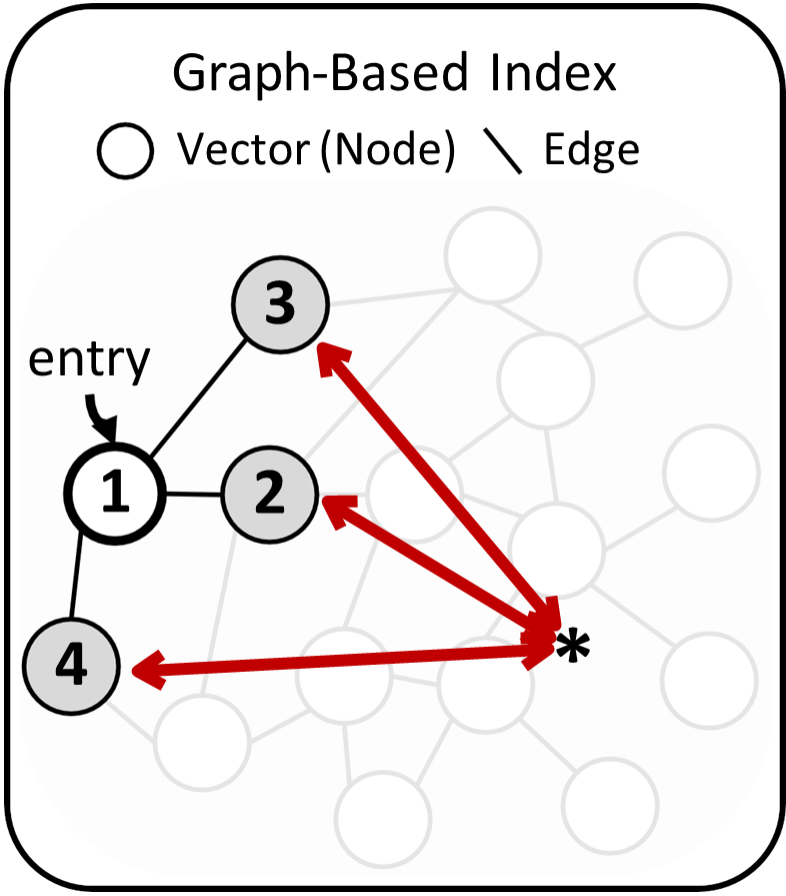
- **Select** the nearest unvisited vector **1** in the *candidate pool*
- **Read** its neighbor IDs **{** 2, 3, 4 **}**



Best-First Search Algorithm

Candidate pool: **{ 1 }**

- **Select** the nearest unvisited vector **1** in the *candidate pool*
- **Read** its neighbor IDs **{** 2, 3, 4 **}**
- **Compare** its neighbors' distances to the target vector ***

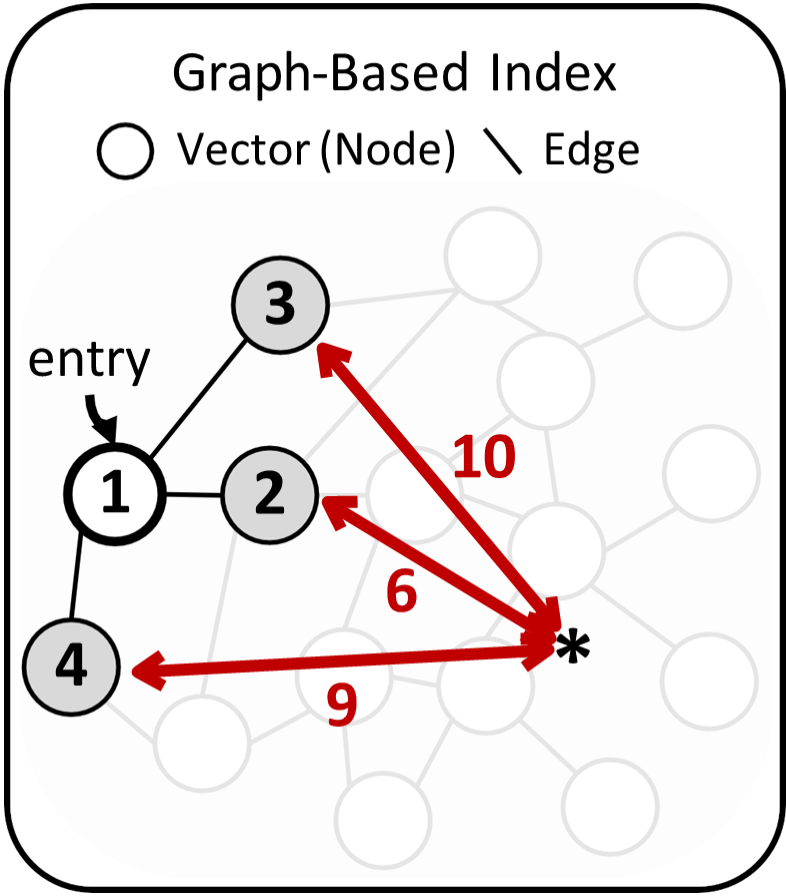


Best-First Search Algorithm

Candidate pool: { 2, 3, 4, ~~1~~ }

- **Select** the nearest unvisited vector **1** in the *candidate pool*
- **Read** its neighbor IDs { 2, 3, 4 }
- **Compare** its neighbors' distances to the target vector *
- **Add** its neighbors to the candidate pool, sort them, mark **1** as visited

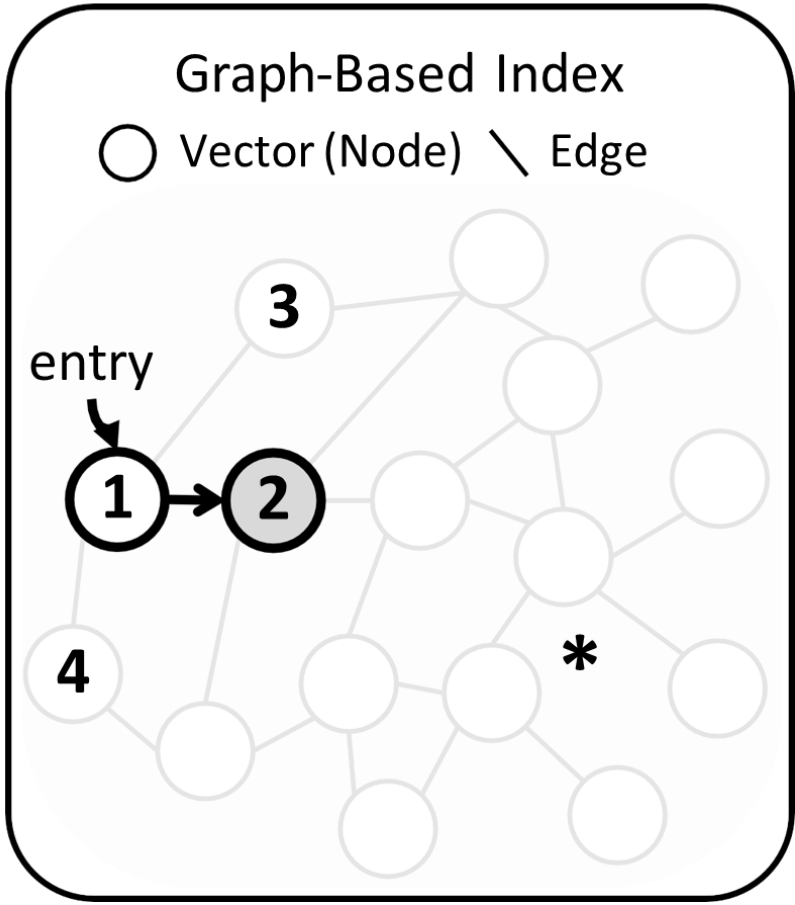
Repeat this process



Best-First Search Algorithm

Candidate pool: **{ 2**, 3, 4, ~~1~~ **}**

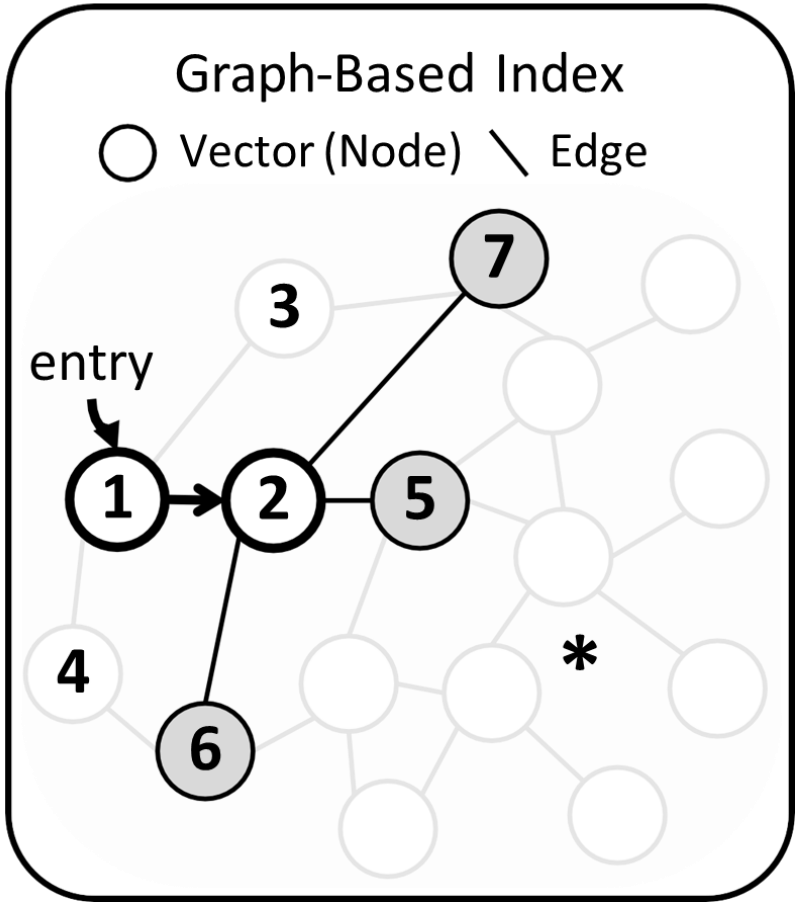
- **Select** the nearest unvisited vector **2** in the *candidate pool*



Best-First Search Algorithm

Candidate pool: **{ 2**, 3, 4, ~~1~~ **}**

- **Select** the nearest unvisited vector **2** in the *candidate pool*
- **Read** its neighbor IDs **{** 5, 6, 7 **}**

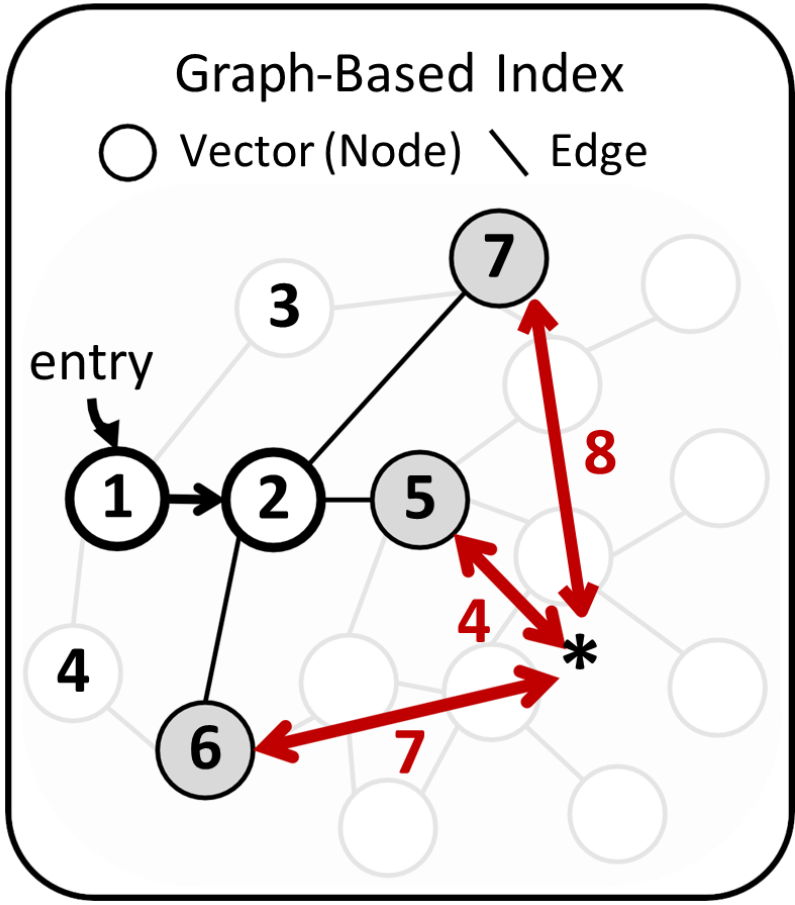


Best-First Search Algorithm

Candidate pool: { 5, ~~2~~, 6, 7, 3, 4, ~~1~~ }

- **Select** the nearest unvisited vector **2** in the *candidate pool*
- **Read** its neighbor IDs { 5, 6, 7 }
- **Compare** its neighbors' distances to the target vector ∗
- **Add** its neighbors to the candidate pool, sort them, mark **2** as visited

Repeat this process

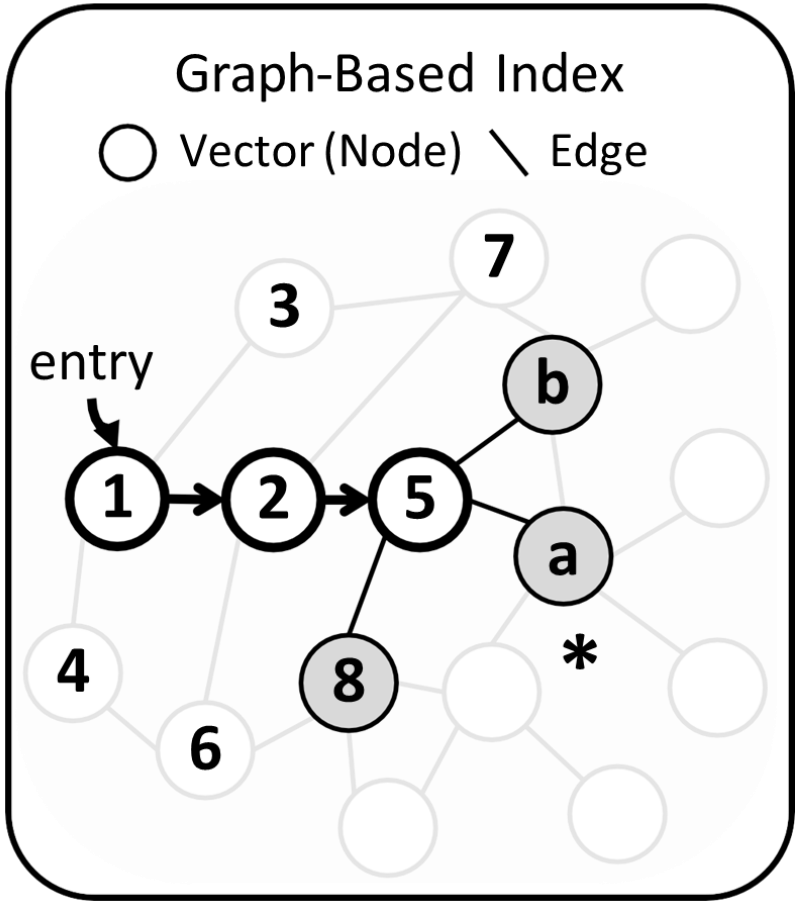


Best-First Search Algorithm

Candidate pool: **{** a, **5**, b, 8, ~~2~~, 6, 7 **}**

- **Select** the nearest unvisited vector **5** in the *candidate pool*
- **Read** its neighbor IDs **{** 8, a, b **}**
- **Compare** its neighbors' distances to the target vector ***
- **Add** its neighbors to the candidate pool, sort them, mark **5** as visited

Repeat this process

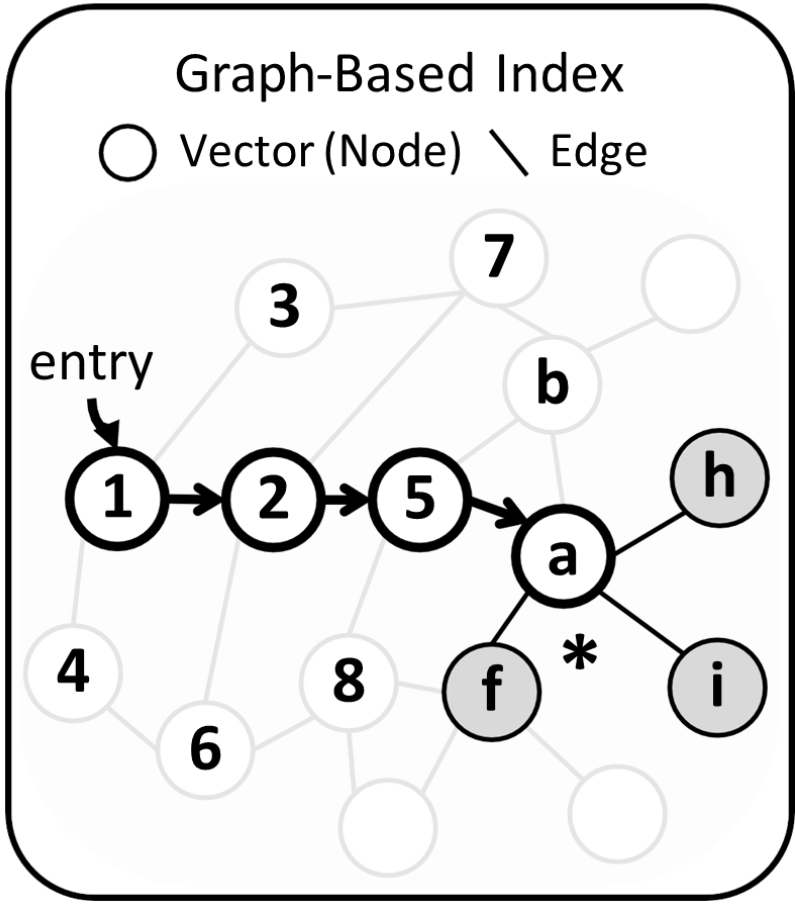


Best-First Search Algorithm

Candidate pool: **{ a**, f, i, h, ~~5~~, b, 8 **}**

- **Select** the nearest unvisited vector **a** in the *candidate pool*
- **Read** its neighbor IDs **{** f, h, i **}**
- **Compare** its neighbors' distances to the target vector ***
- **Add** its neighbors to the candidate pool, sort them, mark **a** as visited

Repeat this process

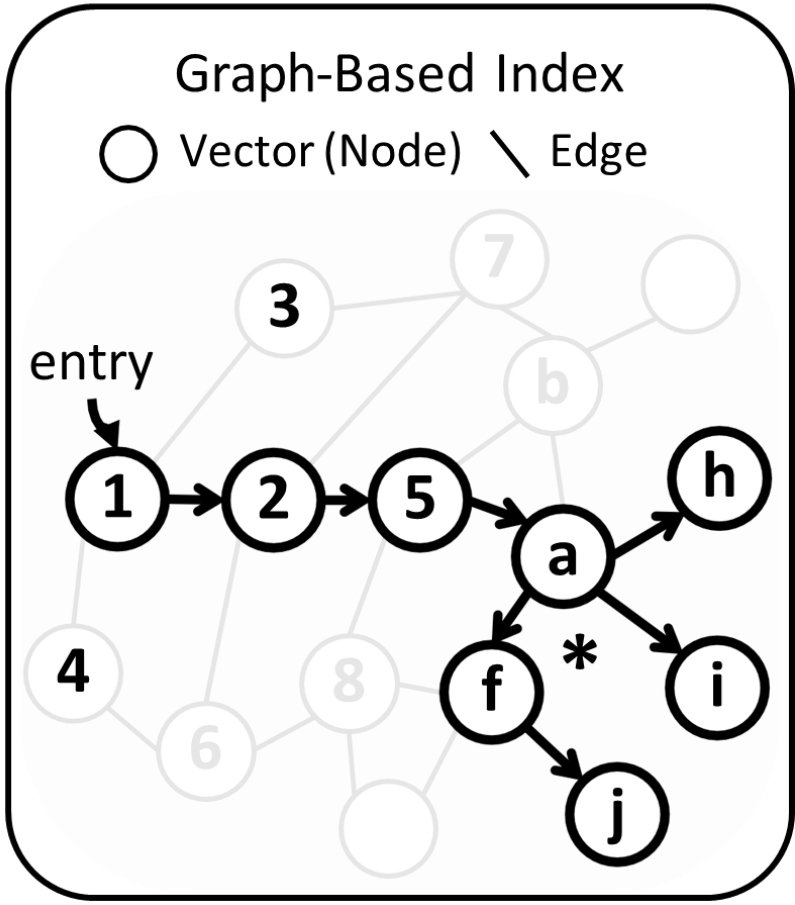


Best-First Search Algorithm

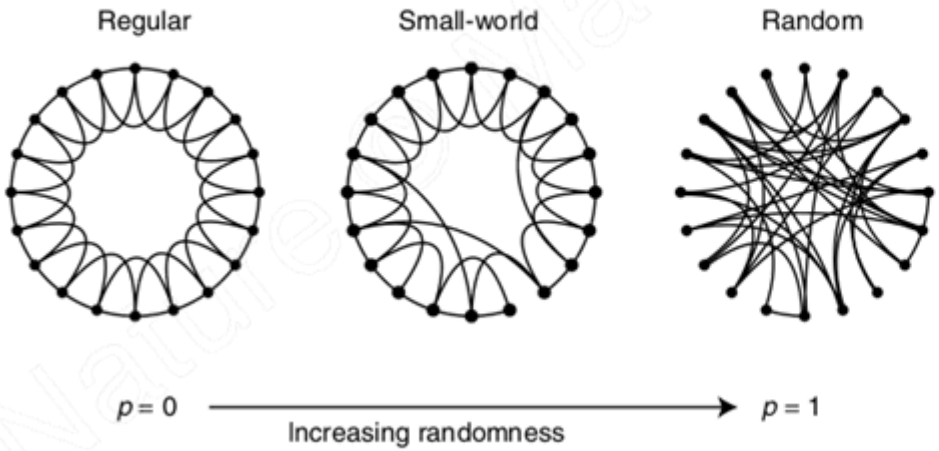
Candidate pool: **{** ~~a, f, i, j, h, 5, b~~ **}**

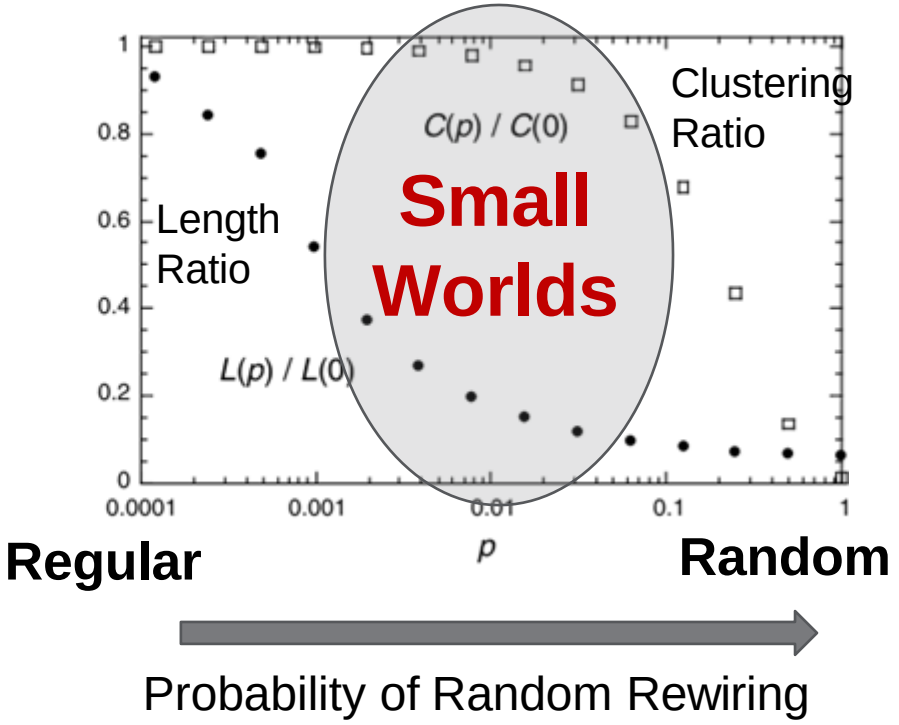
- **Select** the nearest unvisited vector **h** in the *candidate pool*
- **Read** its neighbor IDs
- **Compare** its neighbors' distances to the target vector ***
- **Add** its neighbors to the candidate pool, sort them, mark **h** as visited

Repeat until all vectors are visited



Small-World Graphs



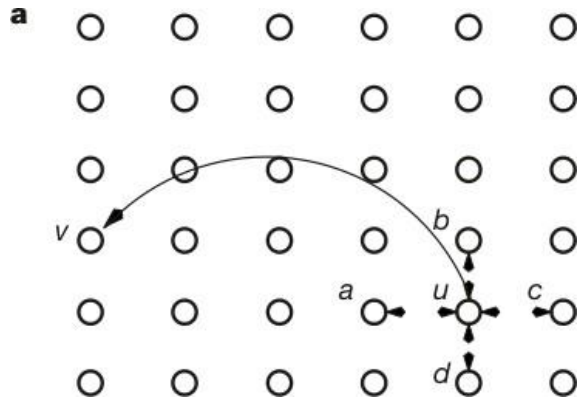


- Small characteristic path lengths (short shortest-paths)
- High clustering (friend of a friend is also my friend)

Figure: Watts and Strogatz, Nature 1998

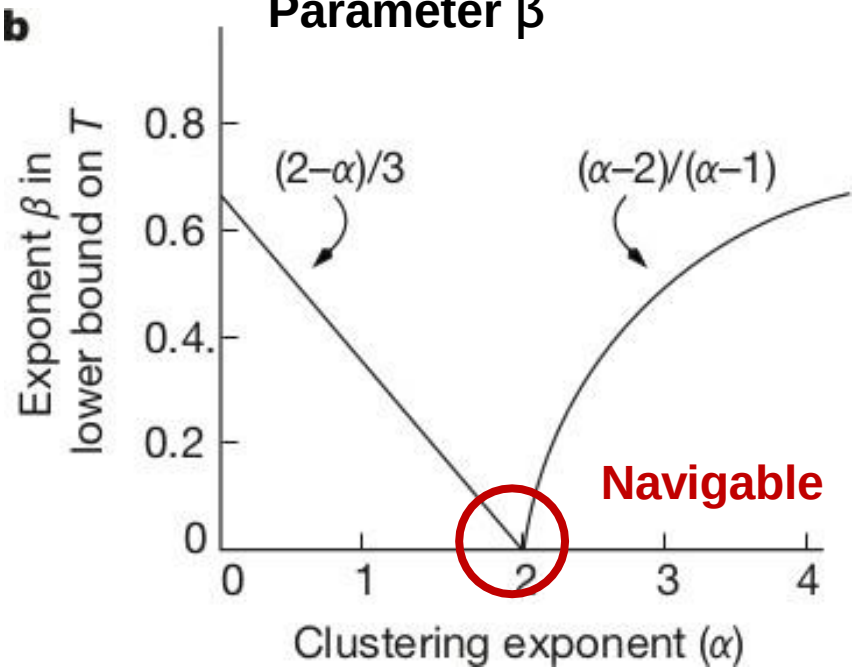
Navigable Small-World Graphs

Random edge probability α



Randomly add an edge from u to v with probability $|u,v|_1^{-\alpha}$

O(N$^{\beta}$) Greedy Search Complexity Parameter β



- Not all small-world graphs permit greedy search

Figure: Kleinberg, Nature 2000

Hierarchical Navigable Small-World Graphs (HNSW)

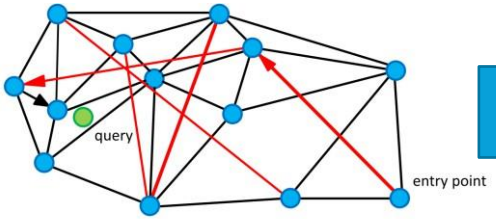


Fig. 1. Graph representation of the structure. Circles (vertices) are the data in metric space, black edges are the approximation of the Delaunay graph, and red edges are long range links for logarithmic scaling. Arrows show a sample path of the greedy algorithm from the entry point to the query (shown green). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

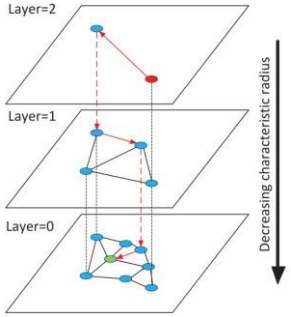
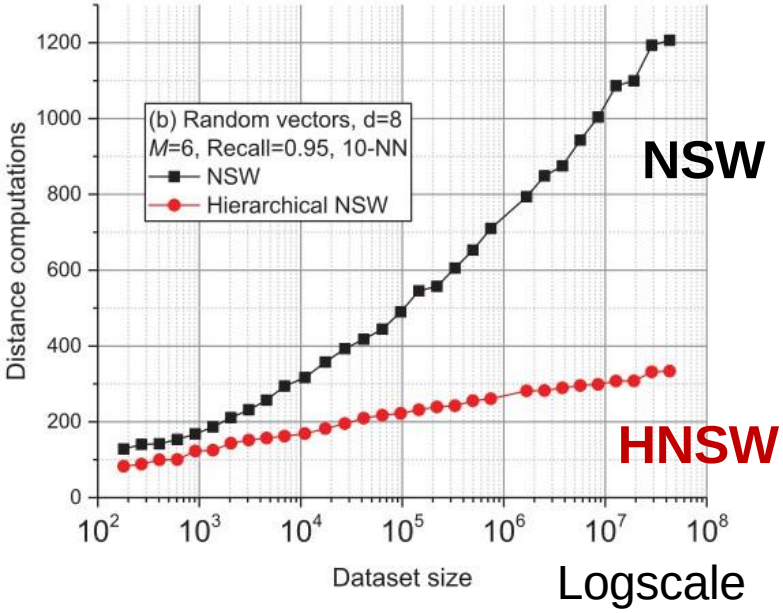


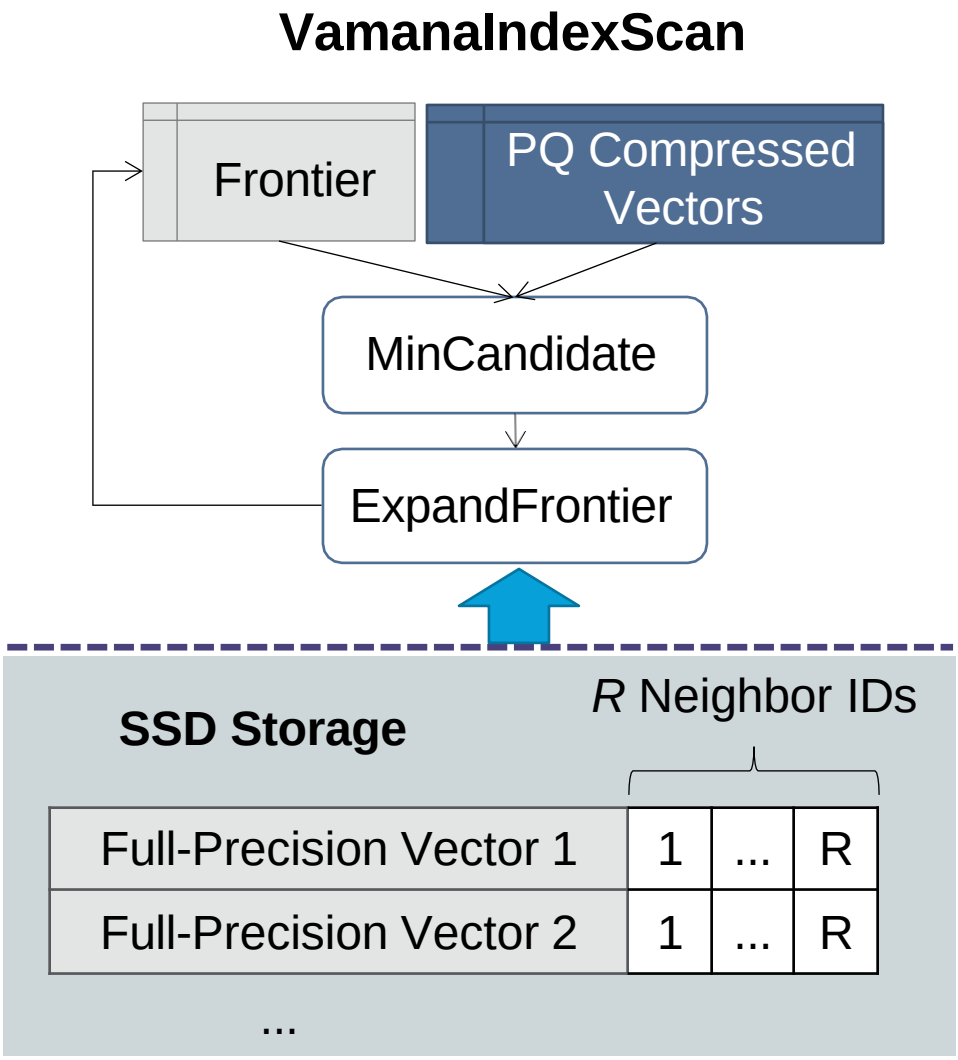
Fig. 1. Illustration of the Hierarchical NSW idea. The search starts an element from the top layer (shown red). Red arrows show directi the greedy algorithm from the entry point to the query (shown green



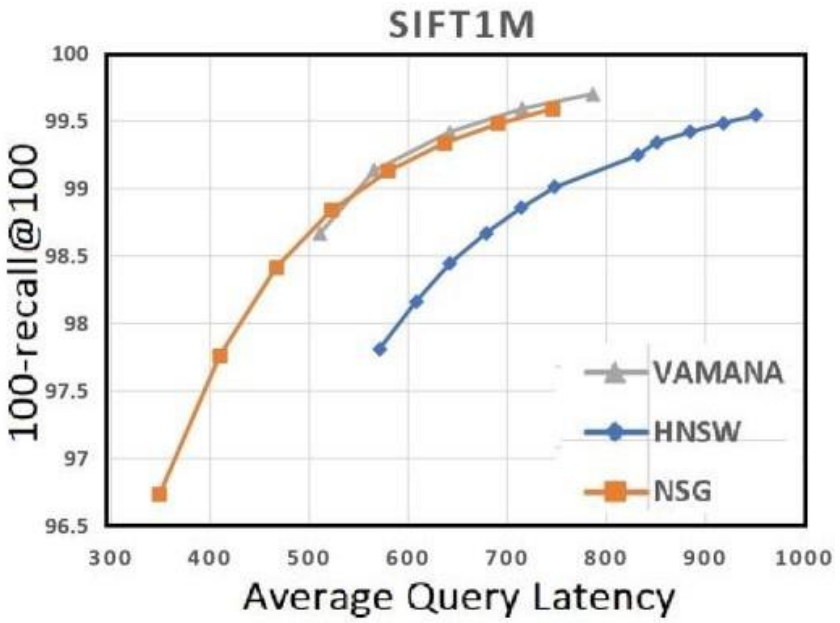
- Simply **inserting vectors one at a time**, connecting it to its k nearest neighbors already in the graph found via search trial, is navigable and small-world
- Hierarchical levels mitigates high out-degrees

Figures: Malkov et al 2014, Malkov & Yashunin 2020

Vamana/DiskANN



- On-disk neighborhoods
- Edge traversal performed in memory using compressed vectors



Summary of Indexes

Index Type	Search Efficiency	Search Accuracy	Write Friendliness	Disk Friendliness
Table-Based		↑	↑	↑
Tree-Based	↑			↑
Graph-Based	↑	↑		↓

Index Type	Construction Efficiency	Storage Efficiency	Ease of Maintenance
Table-Based			↑
Tree-Based			
Graph-Based	↓		

2. Scalable Vector Search Systems

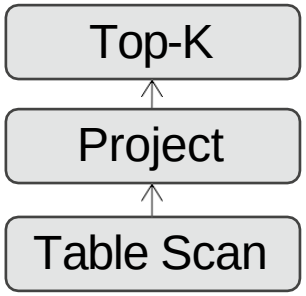
- Filter query
- Hardware Acceleration
- Data Manipulation
- Distributed Processing

Filter Query

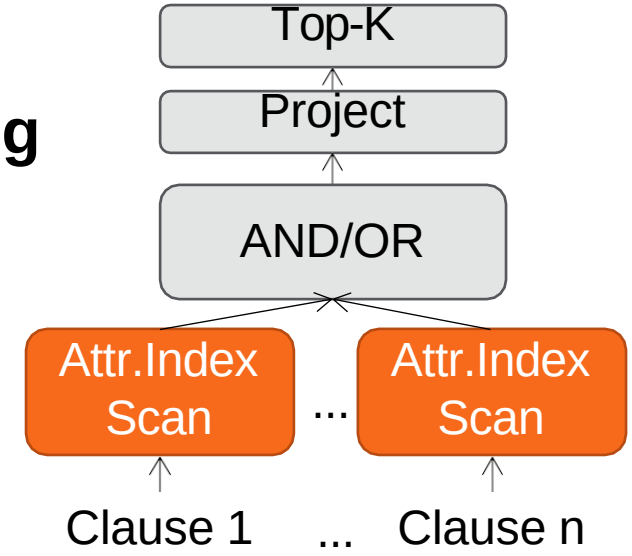
select * from items where price < $10 order
by
simTo(query) limit k

Filter Query

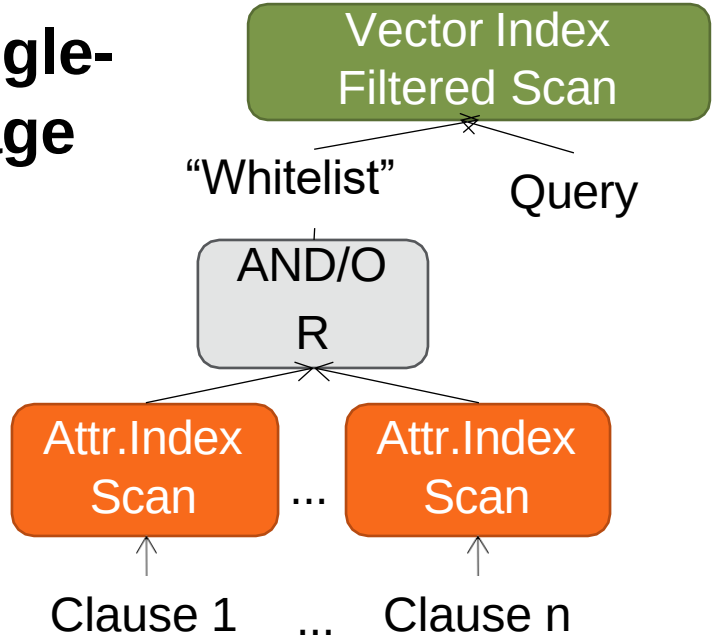
Brute-Force



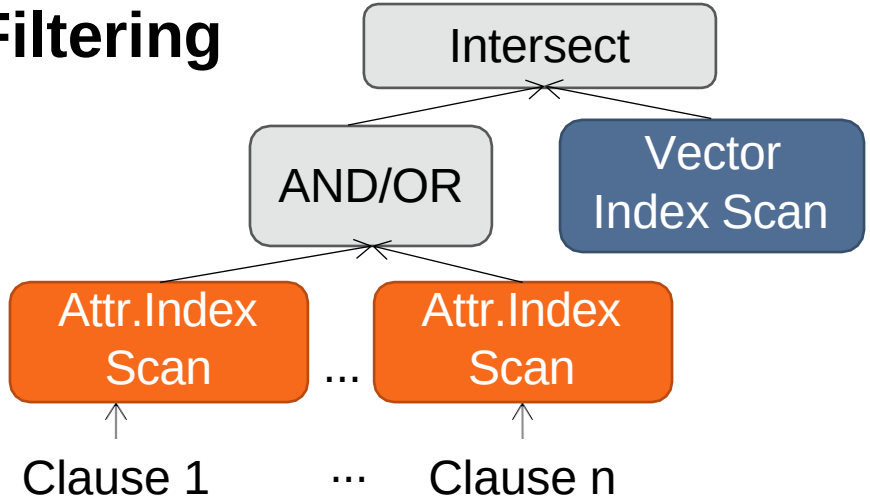
Pre-Filtering



Single-Stage



Post-Filtering



Filter Query

Plan Type	Advantages	Disadvantages
Brute-Force	√ Exact (100% Recall)	X High latency for weak filters
Pre-Filtering	√ Exact (100% Recall), efficient for strong filters	X High latency for weak filters
Post-Filtering	√ Efficient (Native vector search speed & native attribute filter speed)	X Low accuracy risk (e.g. empty intersection). Mitigation: collect (αk) similar vectors, not just k. E.g. ADBV
Single-Stage Filtering	√ No loss of recall, often more efficient than pre-filtering	X Possibly high latency for strong filters. Mitigation for graph-based indexes: Encourage visiting satisfying vectors, e.g. FilteredDiskANN, HQANN, NHQ; Increase reachability, e.g. ACORN; Decrease failures via partitioning, e.g. Milvus

Filter Query

Rule-Based, e.g. Qdrant, Vespa

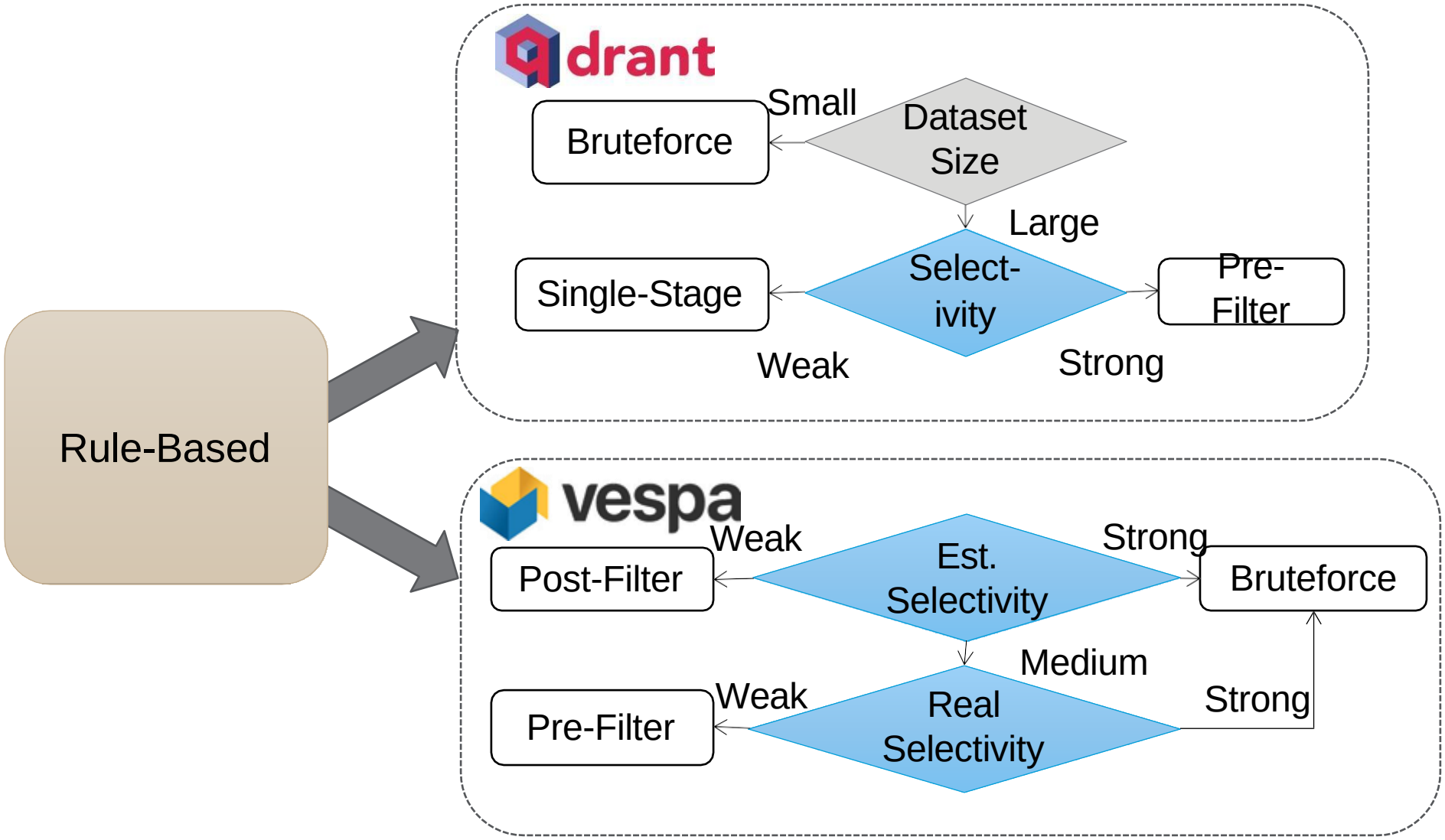
- Simple to implement
- Depends on accurate selectivity estimates

Cost-Based, e.g. ADBV, Milvus

- Select based on a cost model
- Generally more flexible
- Depends on accurate operator cost estimates

Rule-Based Plan Selection

Example: Qdrant and Vespa



Cost-Based Plan Selection

Table 1: Notations used by hybrid query optimization

Notation	Meaning
n	the total number of tuples in database
α	the ratio between n and the number of records satisfying structured predicate
β	the visited subcells ratio during `VGPQ` index searching process in VGPQ Knn Bitmap Scan
γ	the visited subcells ratio during `VGPQ` index searching process in VGPQ Knn Scan
$\sigma_{\{B,C,D\}}$	amplification factors of *ANNS Scan* operators in $Plan\{B, C, D\}$
c_1	the total time cost to fetch a vector and compute pairwise distance
c_2	the total time cost to fetch a `PQ` code and run `ADC`

- Pre-Filter k-NN Scan

$$cost_A = T_0 + \alpha \times n \times c_1$$

- Single-Stage PQ Filtered Scan

$$cost_B = T_0 + \alpha \times n \times c_2 + \sigma_B \times k \times c_1$$

- Single-Stage VGPQ Filtered Scan

$$cost_C = T_0 + \beta \times n \times \alpha \times c_2 + \sigma_C \times k \times c_1$$

- Post-Filter VGPQ Scan

$$cost_D = \gamma \times n \times c_2 + \sigma_D \times k \times c_1$$

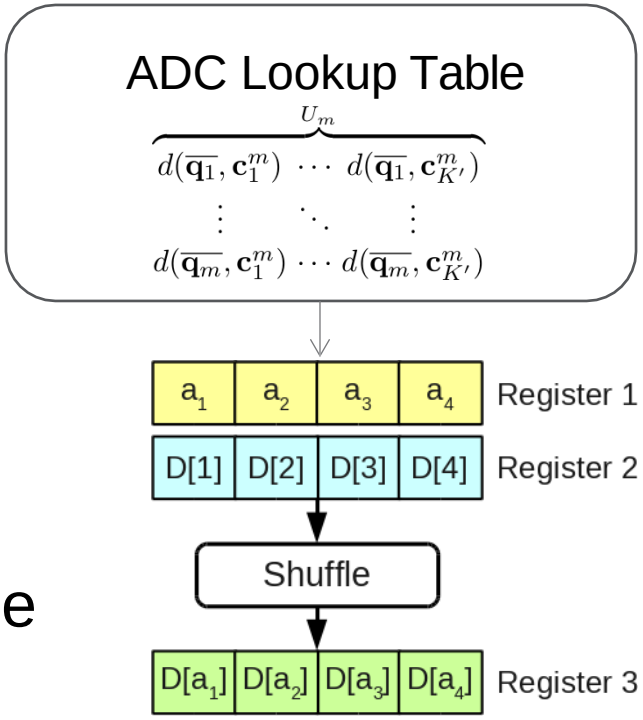
Hardware Acceleration

Data Transfers

- CPU Cache: “Query blocks” keep query vectors cache-resident while assigning threads to data vectors keeps data vectors cache-resident, e.g. Milvus

Data/Task Parallelism

- SIMD/GPU for IVFADC, e.g. Faiss
  - Parallelize lookups by keeping the lookup table inside the SIMD register and simulate lookups via SIMD shuffle (also avoids memory retrieval)
  - Parallelize summations over registers



Data Manipulation

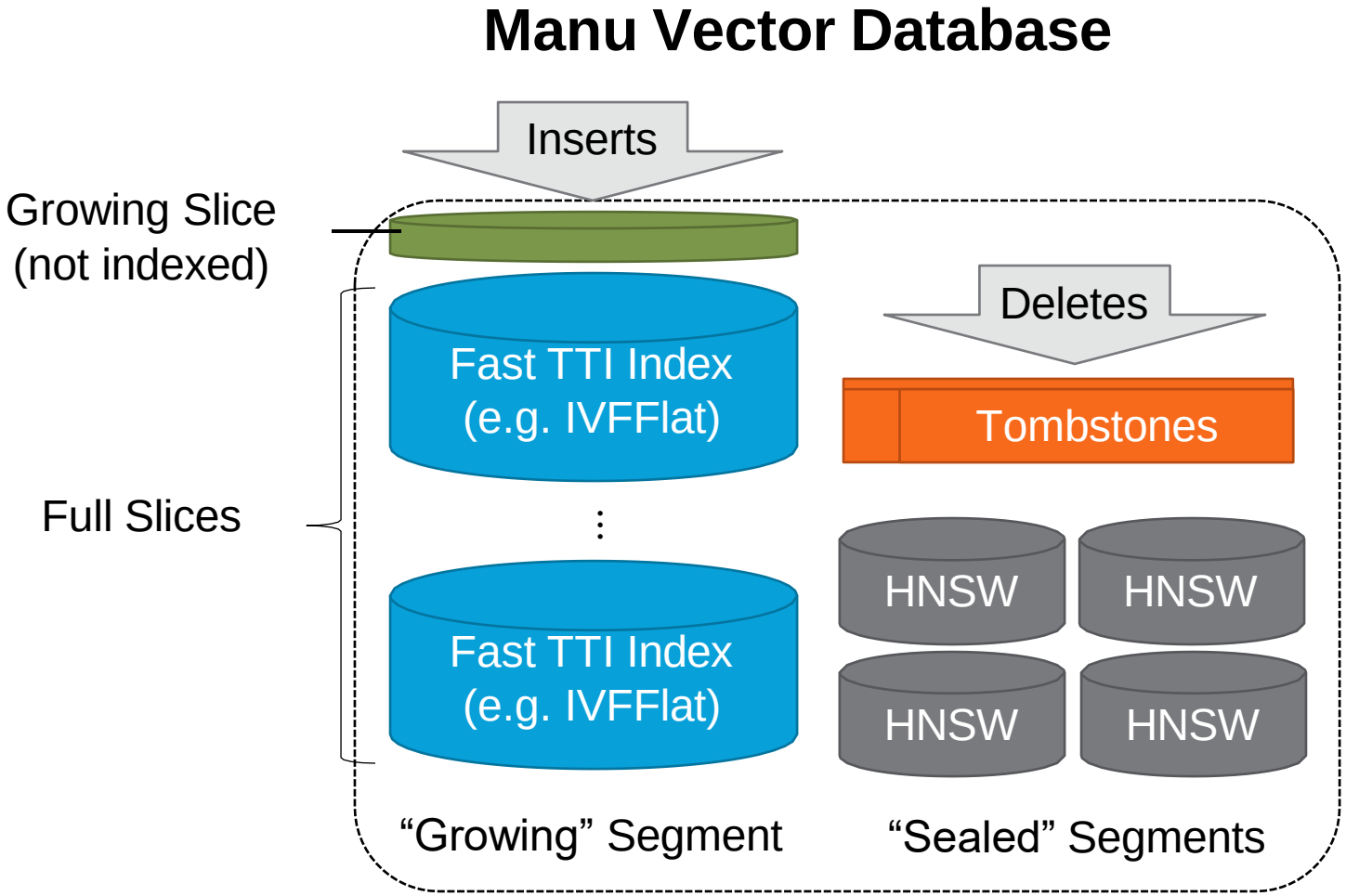
Streaming Updates

- Some indexes support in/up/del, e.g. HNSW
- Vearch: use tombstone deletes to avoid disconnecting the graph + periodic garbage collection

Batched Updates

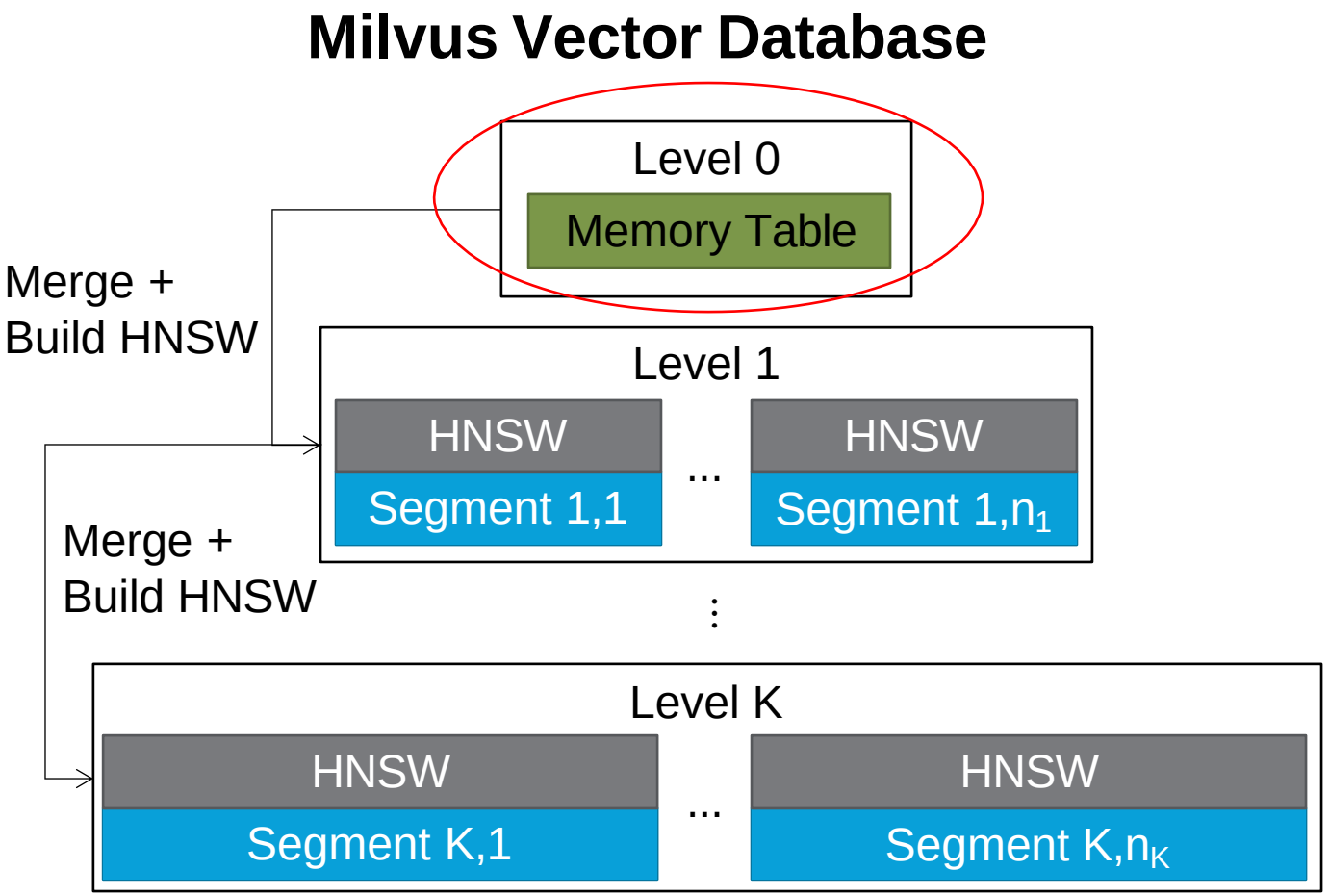
- Perform in/up/del inside a fast-writeable slow-readable structure which also participates in search
- Reconcile into the slow-writeable fast-readable structure at a convenient time

Fast Slices with Slow Segments



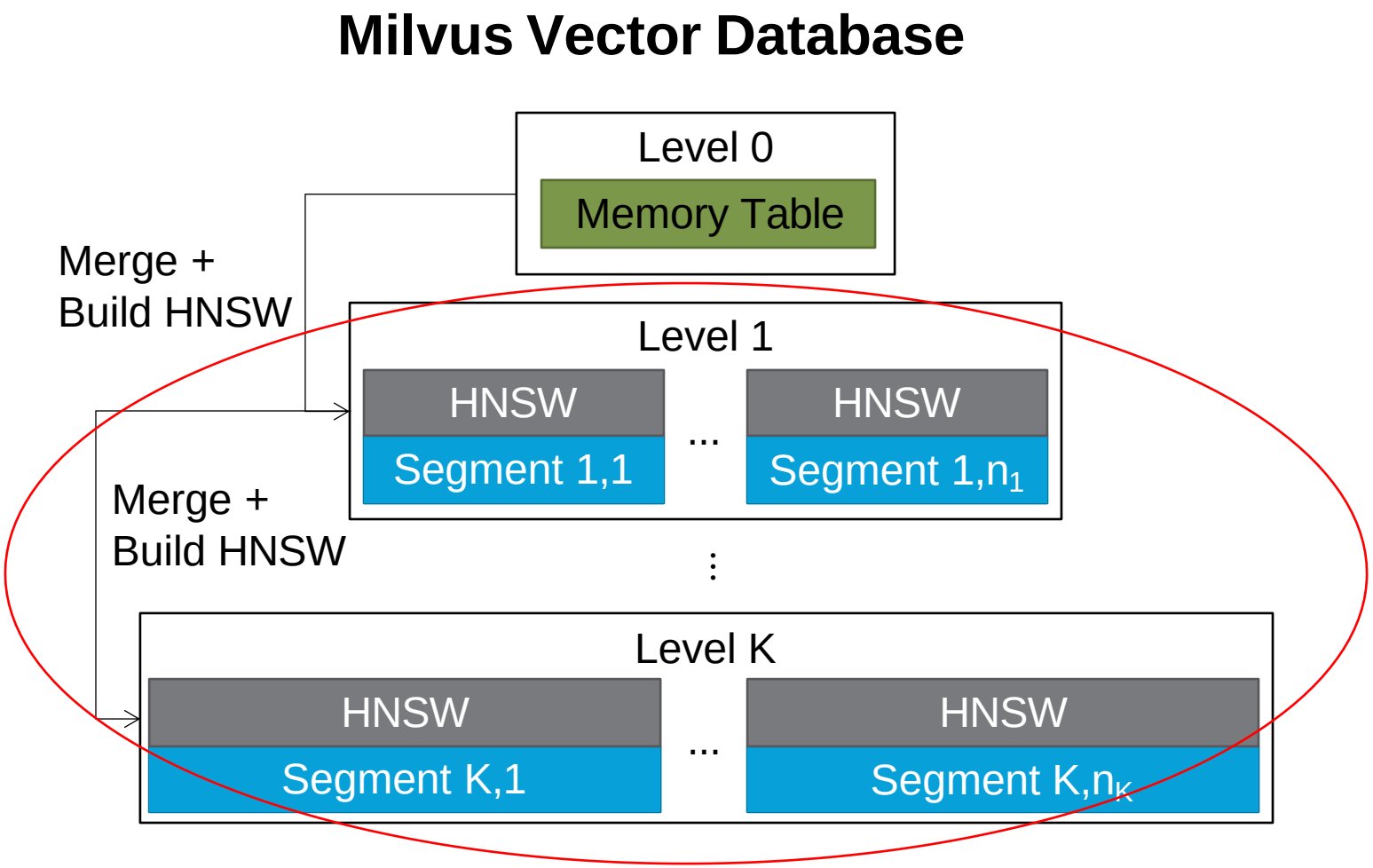
- HNSW is built over the growing segment once full, and then the temporary slices are discarded

Log-Structured Merge (LSM) Tree



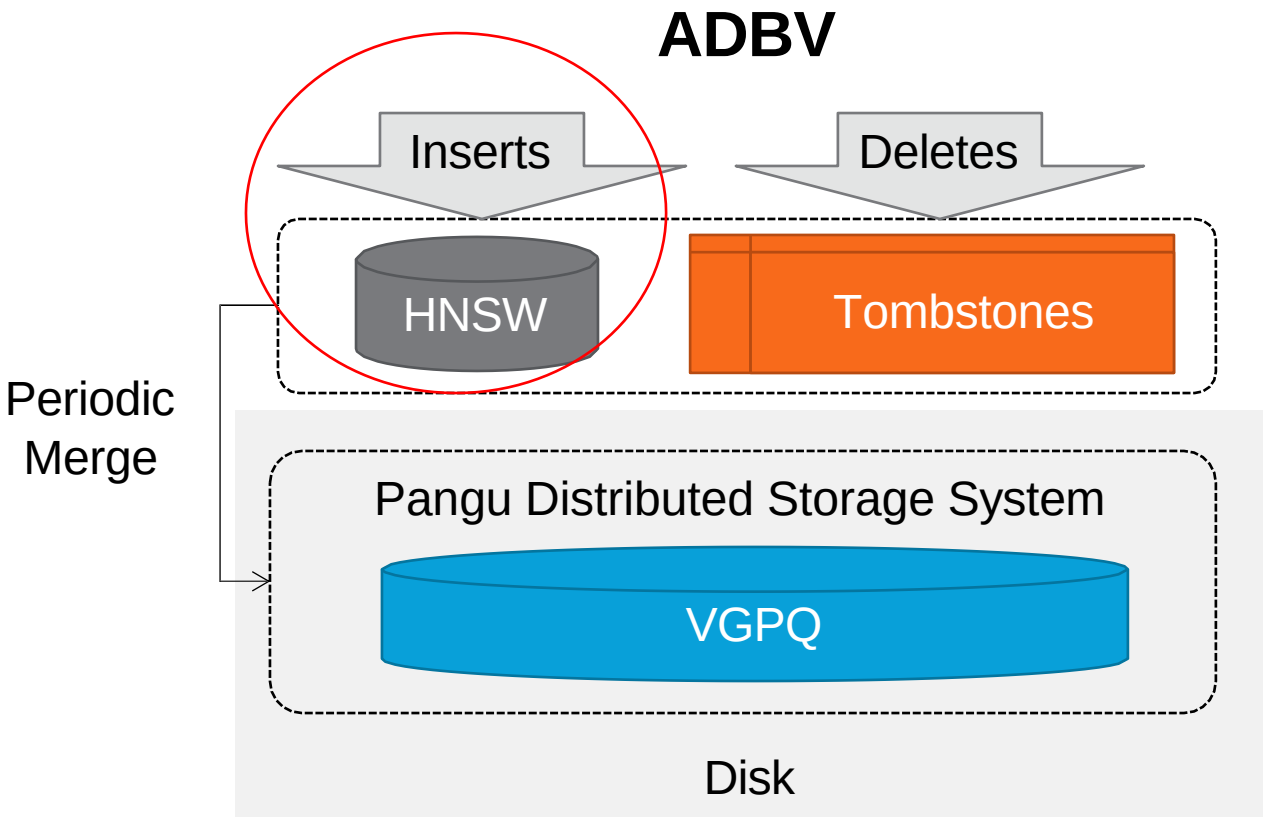
- HNSW built during segment compaction
- Tombstones are reconciled during compaction

Log-Structured Merge (LSM) Tree



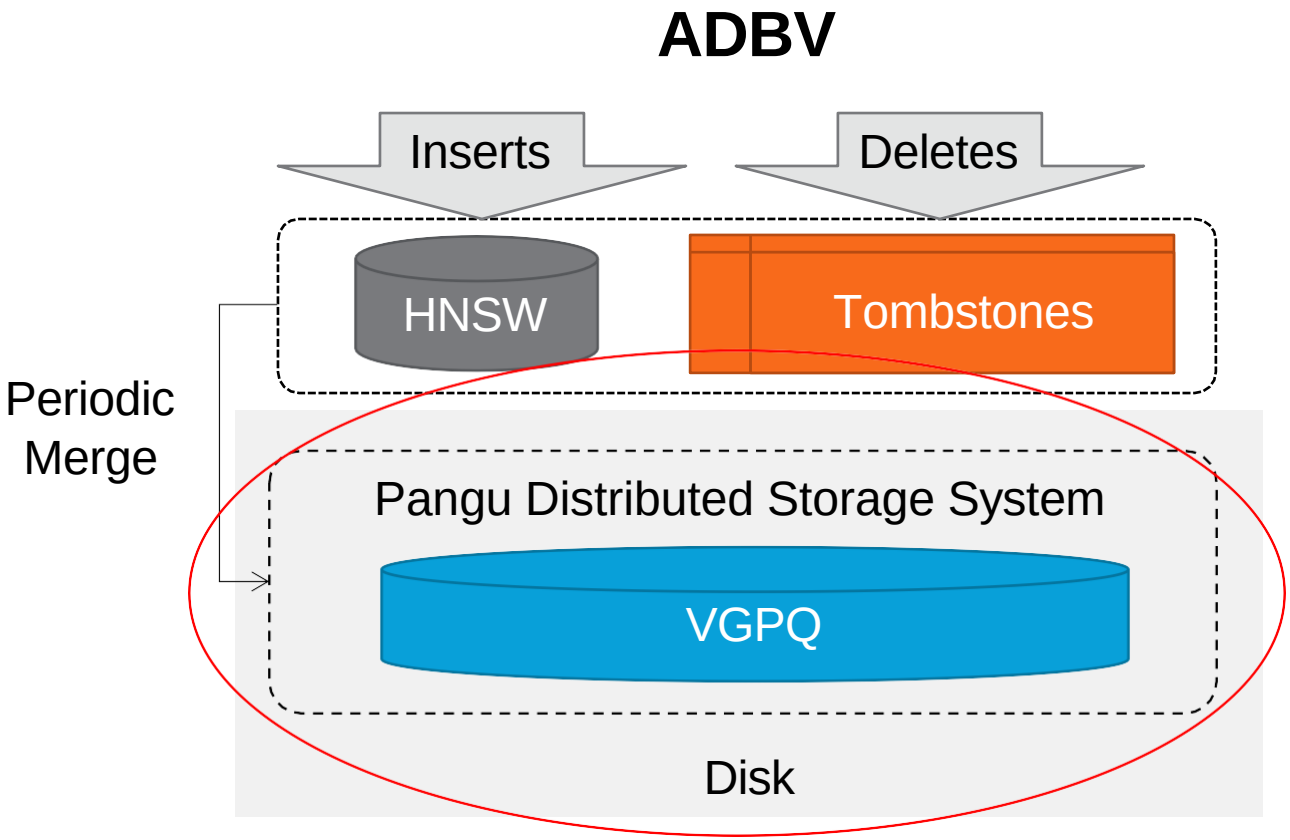
- HNSW built during segment compaction
- Tombstones are reconciled during compaction

In-Memory HNSW with Disk-Resident Index



- Designed for massive TB+ datasets
- HNSW serves as the fast-writeable structure while disk-resident VGPQ is the slow-writeable structure

In-Memory HNSW with Disk-Resident Index



- Designed for massive TB+ datasets
- HNSW serves as the fast-writeable structure while disk-resident VGPQ is the slow-writeable structure

Distributed Query Processing

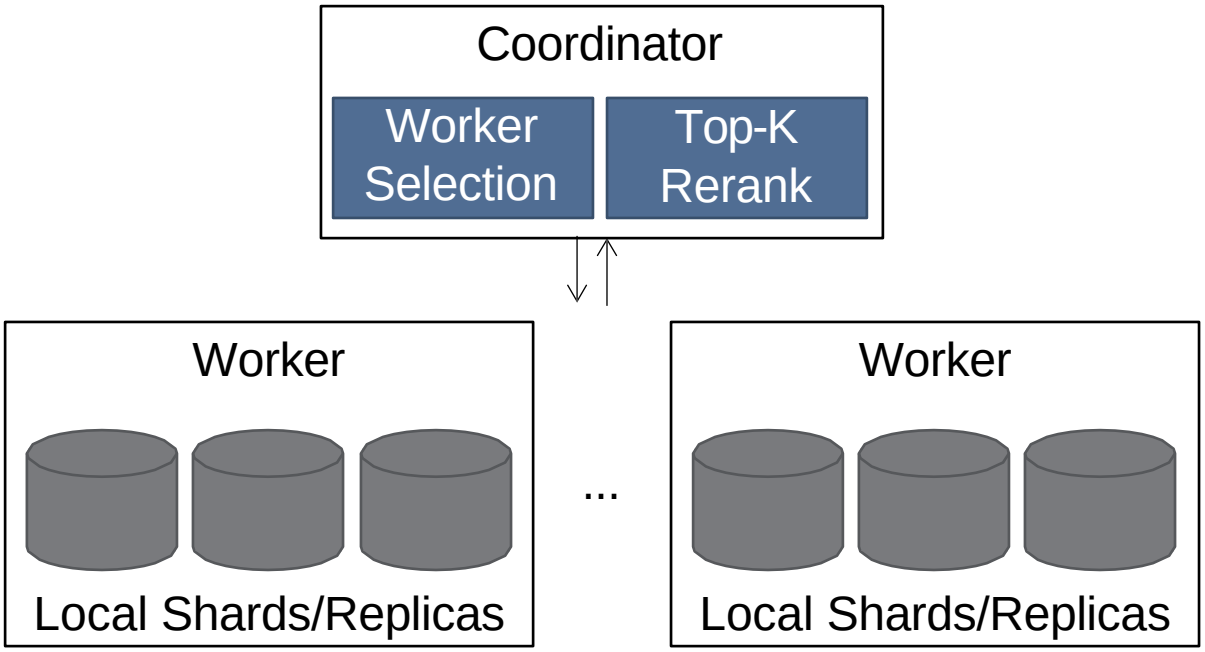
Partitioning

- By attribute if available, e.g. ADBV
- By k-means cluster, e.g. ADBV
- By memory availability, e.g. Vald
- By uniform hashing

Consistency

- Eventual consistency
  - By quorum, e.g. Weaviate
  - By timestamp delta, e.g. Manu
- Strong consistency
  - Concurrent HNSW via internal locks, e.g. Vearch

Scatter-Gather Search



- Hard to know beforehand which workers to select
- k-means partitioning can reduce searched partitions but needs rebalancing

3. Commercial vector systems

Commercial VDBMSs

Native

Extended

Search Engines / Libraries

VDBMS Types and Capabilities

Native Systems

- Mostly-Vector — Pinecone
- Mostly-Mixed — milvus

- Limited query/index types, mainly read-heavy, limited or no predicated queries
- Multiple query/index types, mixed workloads, predicated + attribute-only queries

---

Extended Systems

- NoSQL — 
- Relational — 

- Basic vector search capability, similar to Mostly-Vector systems
- Comprehensive capabilities, similar to Mostly-Mixed systems

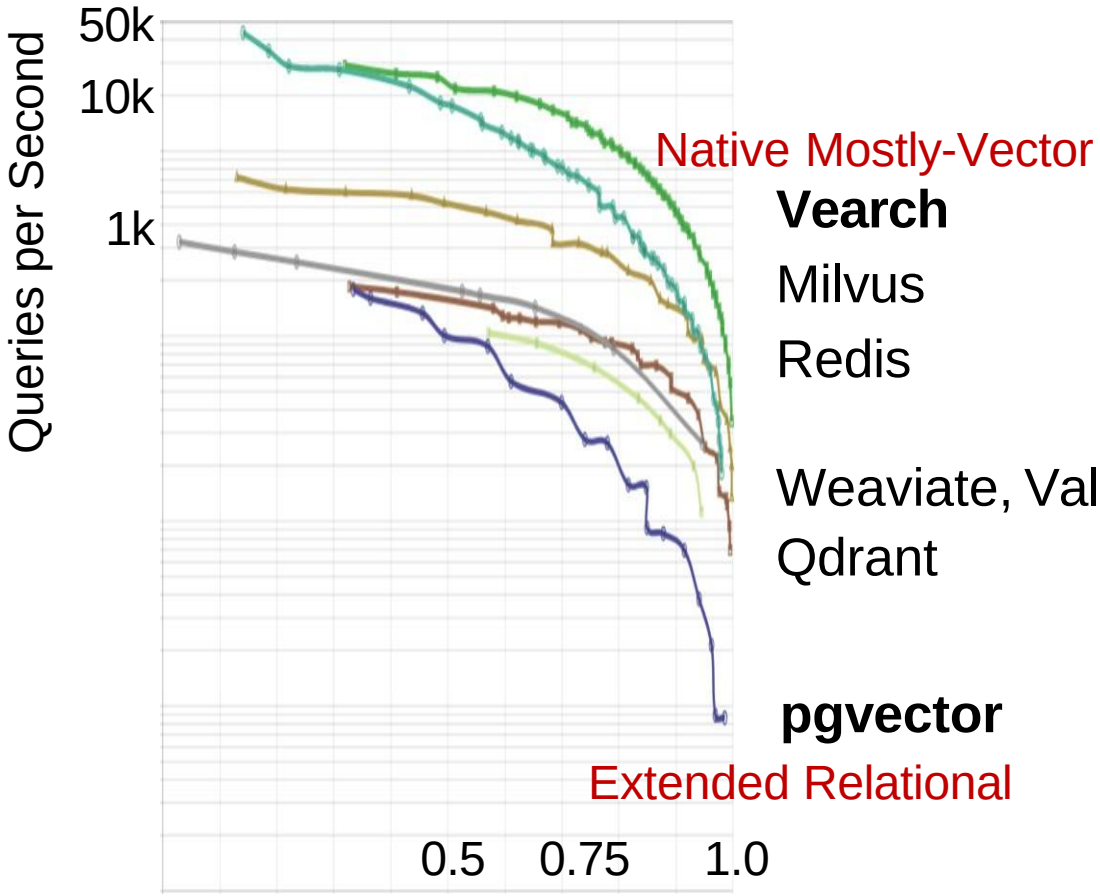
---

Search Engines & Libraries

- Search Engines — APACHE LUCENE
- Libraries — 

- Embedded at application-level
- Offers specific functionality, e.g. a single vector index

VDBMS Performance



Source: ann-benchmarks.com

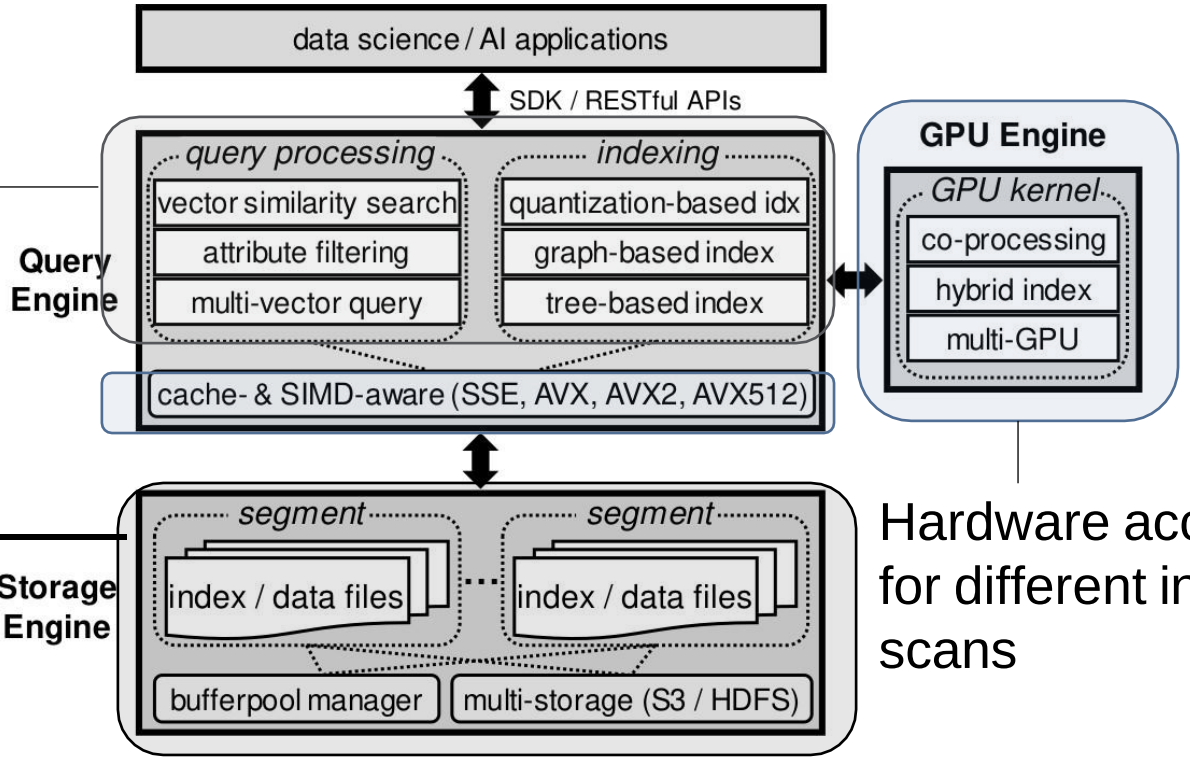
- Native systems “tend to” outperform extended systems
- This view is already being challenged. Zhang et al ICDE’24: “...there is **no fundamental limitation in using a relational database** (e.g., PostgreSQL) to support efficient vector data management”

Native system

Example: milvus

More query capabilities and index types

More sophisticated storage to support fast writes in addition to reads



Hardware accelerator for different index scans

Advantages

√ Many supported query types, can be configured for both read-heavy and write-heavy workloads

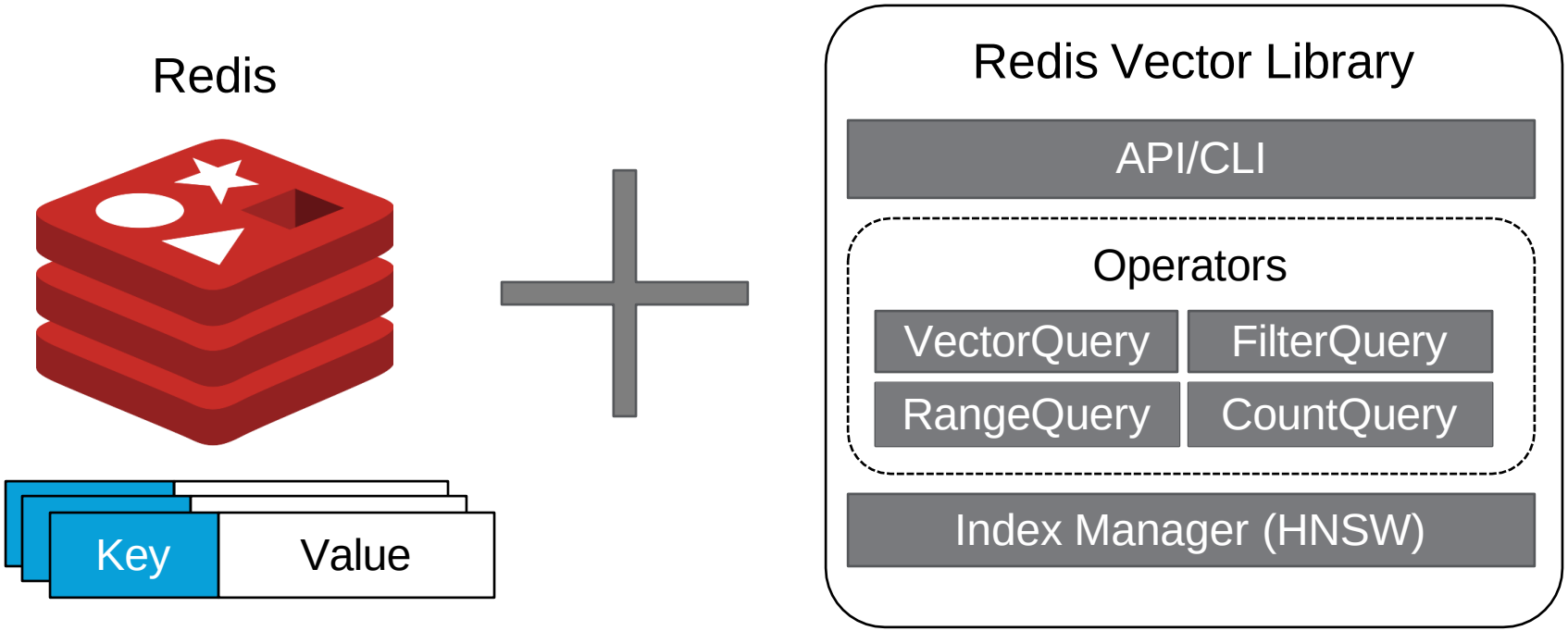
Limitations

X Can be resource-intensive due to more sophisticated storage/recovery

Figure: Wang et al “Milvus: A purpose-built vector data management system”. SIGMOD 2021

Extended NoSQL

Example: RedisVL



Advantages

√ High-performance vector search, similar to Native Mostly-Vector

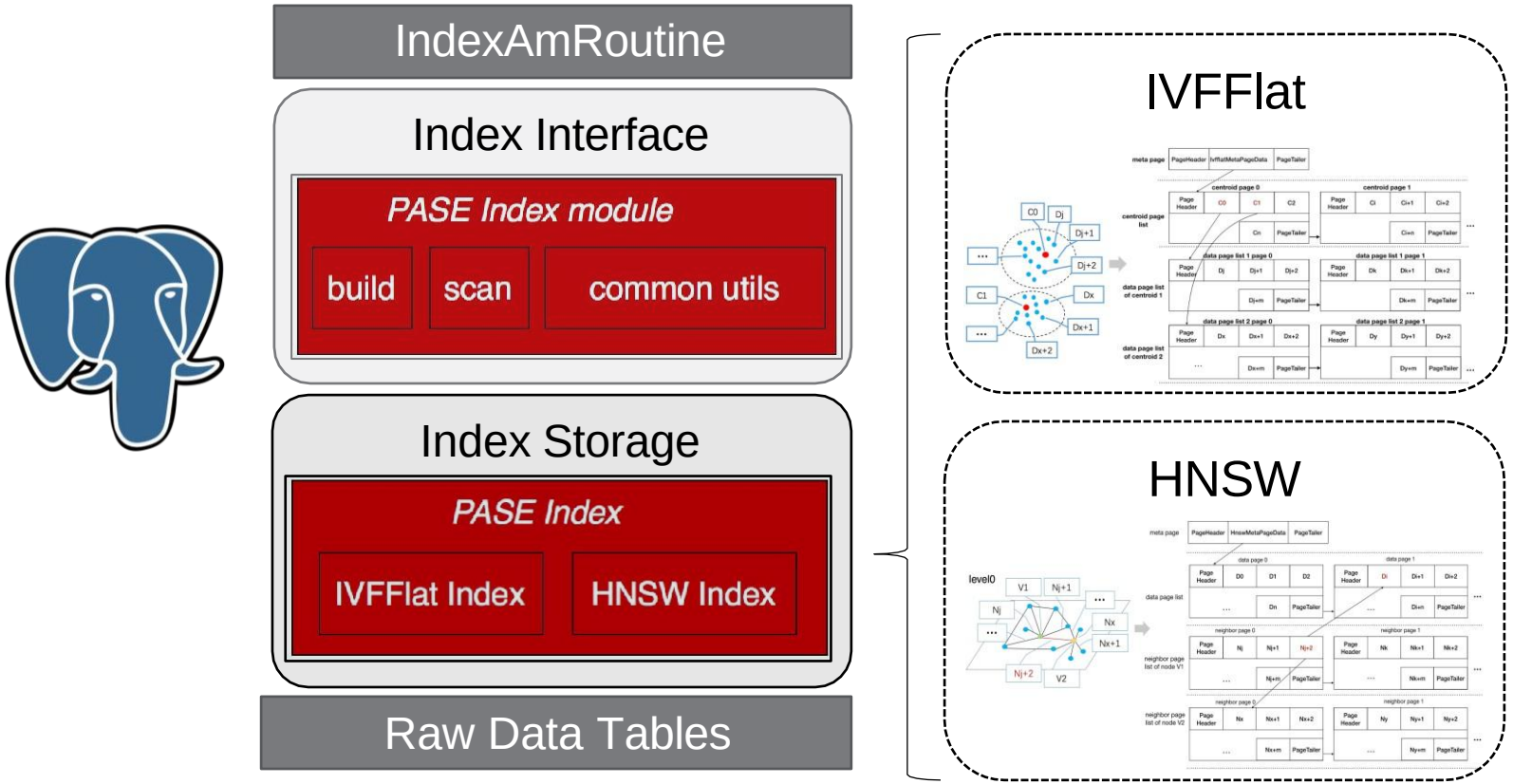
√ Combined vector search + non-vector capabilities

Limitations

X As with Mostly-Vector, performance is tied to specific workload

Extended Relational

Example: PASE



Advantages

√ Adaptable to different types of workloads

√ As with Ext NoSQL systems, offers diverse capabilities

Limitations

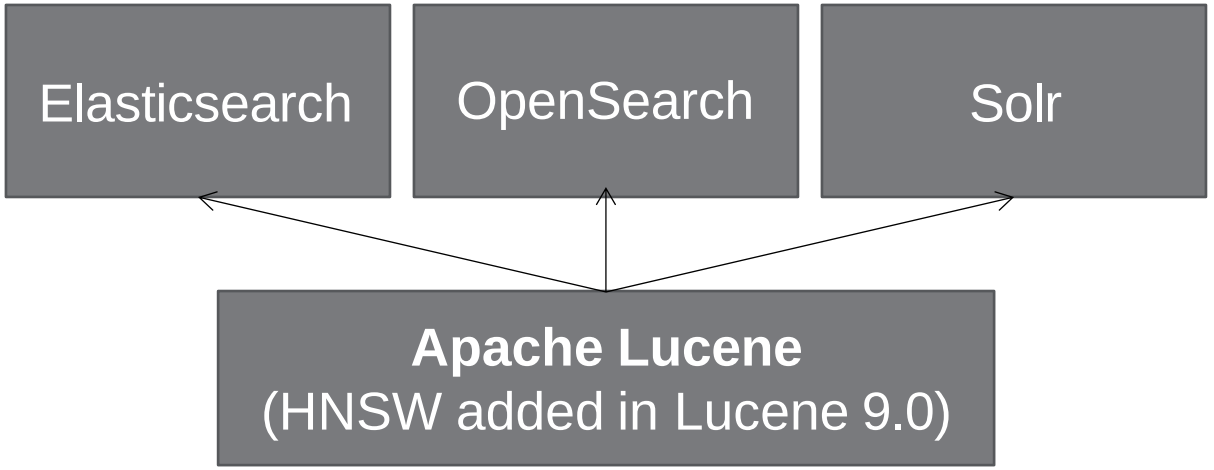
X May suffer performance overhead (e.g. page indirection)

Figure from Yang et al "PASE...". SIGMOD 2020

Search Engines and Libraries

Lin et al “**Vector Search with OpenAI Embeddings: Lucene Is All You Need**” (2023) arXiv:2308.14963

Search Engines



Libraries: Meta Faiss, hnswlib, ANNOY, Microsoft SPTAG, KGraph, E2LSH, FALCONN, etc.

Benchmarks

- Surprisingly few benchmarks
- **ann-benchmarks.com**
  - Real implementations, highly implementation-dependent
- **Li et al TKDE 2020**
  - Idealized implementations

Li et al “Approximate nearest neighbor search on high dimensional data — Experiments, analyses, and improvement.” IEEE Trans. Knowl. Data Eng. 32(8), 1475–1488 (2020)

4. Challenges and Open Problems

Score Design/Selection

A particular score may not return maximally relevant results, even under high recall:

"The total prod [negative user feedback] rate are 7.3%, 32.6%, and 43.1% at top 1, 3, and 5... roughly 70% are generated by the EBR node during the retrieval stage"

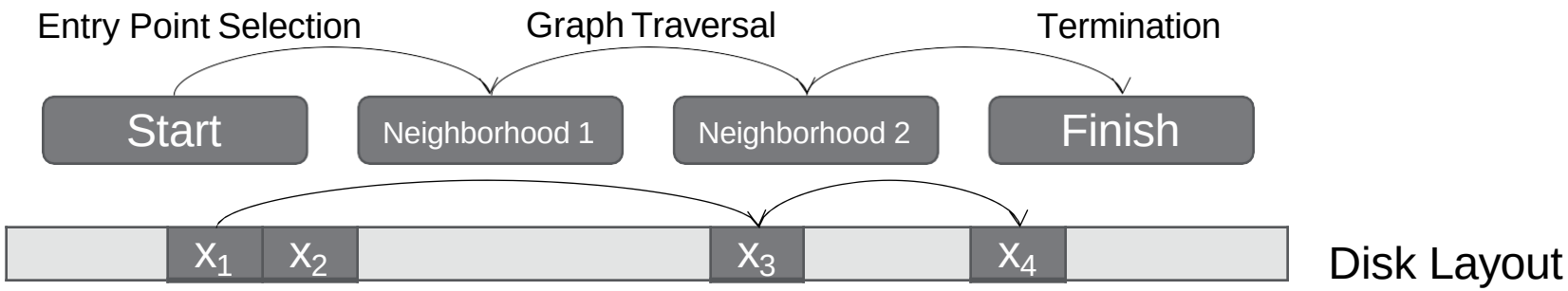
Table 1: Distribution of EBR failures

Failure reasons	Percentage	Category
irrelevant result (fuzzy text match)	53%	junkiness
Location mismatch	18%	junkiness
Language mismatch	4%	junkiness
Misinformation	10%	integrity
Untrustworthy results	10%	integrity
Offensive results	5%	integrity

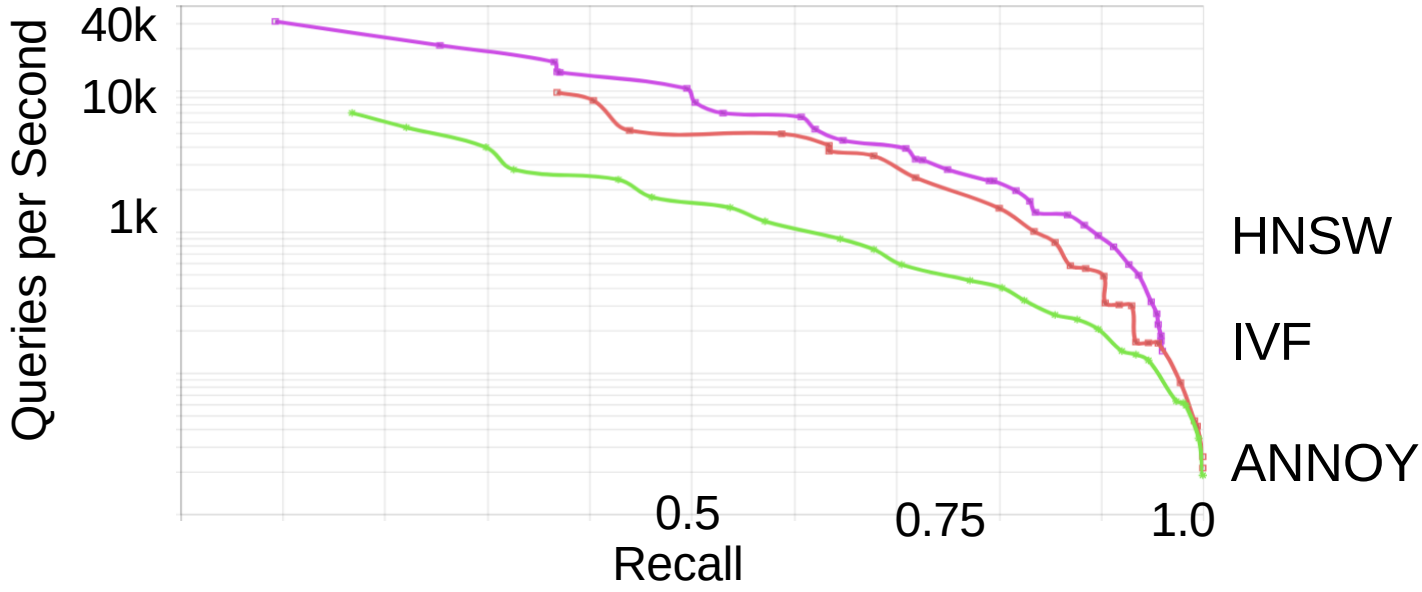
Wang et al "Integrity and junkiness failure handling for embedding-based retrieval: A case study in social network search". SIGIR 2023

Disk-Friendly/Distributed Indexes

Graphs are slow for disk-resident datasets

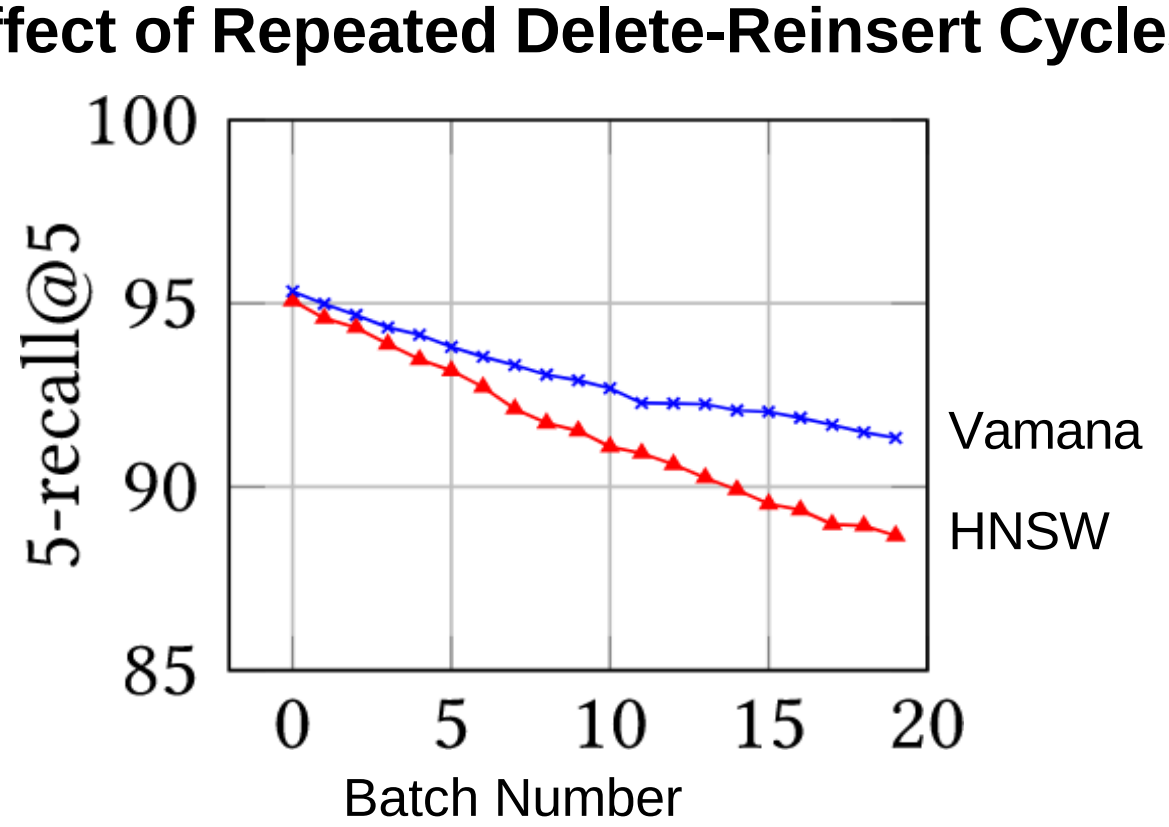


Meanwhile, **trees/tables** are disk-friendly but have worse QPS/recall



Update-Friendly Graphs

Accuracy degradation following series of updates



Singh et al “FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search” 2021. arXiv 2105.09613

Thanks!